Учебная модель capability-делегирования

в стиле Zircon (Fuchsia OS) средствами Isabelle/HOL

redbeardster@gmail.com

2026

Аннотация

В работе построена *учебная* формальная модель фрагмента capability-механизма в стиле микроядра Zircon (Fuchsia OS) в Isabelle/HOL. Доступ к объектам ядра в Zircon идёт через дескрипторы с масками прав; ключевой вопрос делегирования — не получает ли получатель при передаче прав сверх источника. Мы доказываем отсутствие такой эскалации для композиции *transfer_via_channel* (успешный `write`/`read` при *пустой* очереди получателя) на всех состояниях, достижимых из корректного старта (`state_wf`). Той же моделью показана граница гарантии: гипотетический привилегированный poke в store VMO (в духе обхода мимо шлюза прав, а не полная модель `ZX_RSRC_KIND_MMIO`) ломает изоляцию содержимого, хотя проверки capability на syscall-пути остаются на месте. Модель намеренно упрощена и не претендует на верификацию промышленного Zircon.

Соглашения о терминологии. В тексте мы сохраняем английские термины, устоявшиеся в security-литературе, но при первом употреблении даём русский перевод и краткое пояснение в скобках. Ключевые термины: *capability* (полномочие) — объект-носитель права доступа к ресурсу ядра; *handle* (дескриптор) — слот в таблице процесса, ссылающийся на capability; *refinement* (уточнение) — доказательство того, что низкоуровневая реализация является допустимым поведением высокоуровневой спецификации; *attenuation* (ослабление, урезание прав) — свойство передачи не давать получателю больше прав, чем было у источника; *capability-маска* (rights mask) — множество атомарных прав у конкретного capability; *witness* (свидетель) — конкретный объект или сценарий, подтверждающий выполнимость утверждения; *negative witness* (негативный свидетель) — контрпример, опровергающий утверждение; *syscall* (системный вызов) — вызов ядра из пользовательского процесса; *VMO* (Virtual Memory Object, виртуальный объект памяти) — разделяемый объект памяти ядра Zircon; *MMIO* (Memory-Mapped I/O) — отображённый в адресное пространство доступ к регистрам устройств; *сессия* (Isabelle session) — именованный набор теорий, который Isabelle проверяет одной командой сборки (у нас `isabelle build -d . Zircon` и шесть теорий `Portable_Capability`, `Zircon`, `Zircon_Lattice`, `Zircon_Traces`, `Zircon_MMIO`, `Zircon_Code`); это единица сборки proof assistant'а, а не «сеанс работы за компьютером» и не HTTP-сессия.

Граница модели. Наша модель намеренно упрощена и не претендует на верификацию промышленного ядра Zircon в объёме проектов класса seL4. Это учебная формализация фрагмента логики прав, а не полный Zircon API. Наш `zx_channel_write` — прообраз `zx_channel_write_etc` с одной записью `zx_handle_disposition_t(ZX_HANDLE_OP_MOVE, rights = req)`: там `rights` — «desired final rights (not a mask)», и запрос сверх прав источника даёт `ZX_ERR_INVALID_ARGS` — у нас проверка *req* $\subseteq h_rights$. Это *прообраз* одного `MOVE`, а не тождество: у `write_etc` при допустимом запросе получатель берёт ровно `req` («desired final rights»), у нас затем *reduce_rights_wf* может дописать `READ` по учебной зависимости (см. ниже); нет байт сообщения, нет массива handles и нет `DUPLICATE` в disposition. При ошибке у нас `None` не меняет состояние. У обычного `zx_channel_write` handles сбрасываются и при отказе; у `write_etc` с `ZX_HANDLE_OP_MOVE` источник *тоже* закрывается (эквивалент `zx_handle_replace` затем write, «always closed»). Учебный freeze-on-`None` — не семантика ядра. *create_channel_pair* склеивает `zx_channel_create` (оба

конца — создателю) с доставкой второго конца в другой процесс; композиция *transfer_via_channel* дополнительно требует пустую очередь получателя — в Zircon write в непустую очередь допустим. MMIO-обход в §9 — гипотетический poke в `vmo_store` по `obj_id` (тип `Hardware` его не ограничивает), а не путь через `ZX_RSRC_KIND_MMIO`/BTI. Зависимость `WRITE`⇒`READ` (и `EXECUTE`⇒`READ`) учебная: в Zircon эти биты независимы, ядро `READ` само не дописывает. Замыкание копирует `READ` у уже корректного источника, а не чеканит новое право (подробнее — §8.5). Два конца канала с `channel_peer`, проверка `DUPLICATE` и симметрия пиров как инвариант трассы в модели есть. Полный перечень отличий — по ходу §3–§5 и в теориях.

1. Введение

Контроль доступа к объектам ядра относится к числу механизмов, ошибки в которых непосредственно ведут к уязвимостям класса Local Privilege Escalation. Традиционные средства обеспечения качества — модульное тестирование, обзор кода, фаззинг — эффективно выявляют конкретные дефекты реализации, однако не устанавливают общезначимых свойств вида «для всякого состояния, достижимого из корректного старта, передача capability не увеличивает множество прав». Формальная верификация восполняет этот пробел, но часто ассоциируется исключительно с крупномасштабными проектами (например, seL4 [3, 4]), требующими многолетних усилий специализированных групп.

Что мы называем эскалацией. Прежде чем переходить к формализации, зафиксируем рабочее определение. *Эскалацией прав* мы называем ситуацию, когда после операции у получателя появляется право, которого у источника не было: $h_rights(\text{получателя}) \not\subseteq h_rights(\text{источника})$. Требование «отсутствия эскалации» означает: такого состояния не существует *ни при каком* сценарии из корректного начального состояния (`state_wf` и трасса `reachable`) — то есть это утверждение обо всех *достижимых* состояниях модели, а не о конкретном тесте и не о произвольной HOL-записи с «битой» маской. Именно это свойство доказывается ниже для передачи capability через канал — в форме композиции *transfer_via_channel*, которая дополнительно требует пустую очередь получателя (см. границу модели); у одиночного *zx_channel_write* пустота не нужна.

Цель настоящей статьи — показать, что *базовые* архитектурные инварианты capability-модели допускают компактную формализацию и автоматизированную проверку в Isabelle/HOL без построения полной семантики ядра. На учебной модели в стиле Zircon доказаны, в частности:

- *channel_transfer_no_escalation* — нет эскалации при передаче через канал;
- *channel_write_requires_transfer* — без `TRANSFER` отправка невозможна;
- *channel_write_moves_sender_handle* — move у отправителя;
- *vmo_write_denied_without_write_right* — запись в VMO без `WRITE` отвергается;
- *mmio_leak_witness_true* — привилегированный MMIO-обход ломает системную изоляцию store при живом capability-шлюзе;
- *zx_handle_duplicate_no_escalation* — то же сокращение прав (meet + замыкание) при дублировании handle.

Предполагается знакомство читателя с функциональным программированием; специальная подготовка в области математической логики не требуется.

Отношение к seL4. Проект seL4 [3, 4] даёт полный *refinement* (уточнение — пошаговое доказательство того, что каждая низкоуровневая реализация является допустимым поведением высокоуровневой спецификации) от абстрактной спецификации до бинарника. Настоящая работа — иного масштаба и цели: компактное доказательство того же *типа* свойства (*no rights escalation* — отсутствие эскалации прав, то есть получатель не приобретает прав, которых не было у источника) для узкого фрагмента системных вызовов и явный контрпример к системным обещаниям при обходе через MMIO. Помимо самого результата — полной верификации ядра — seL4 даёт переносимую методологию: *refinement*-стек из нескольких уровней, архитектуру под *tractability* (податливость формальному доказательству: ядро спроектировано так, чтобы его инварианты вообще допускали машинную проверку) и доказательство *access-control enforcement* (принудительное соблюдение контроля доступа: политика не только формулируется, но и гарантированно применяется на каждом переходе). Наш фрагмент не воспроизводит эту методологию целиком; он выделяет один её слой — монотонное *attenuation* (ослабление, урезание прав при передаче: получатель не может приобрести прав больше, чем было у источника) capability-масок — и показывает, что даже этот слой даёт содержательные машинно проверяемые гарантии без построения полной семантики ядра. Разница не в «меньшей строгости», а в назначении: seL4 — для сертификации ядра; наша модель — для *ранней* проверки архитектурных решений и обучения языку формальных инвариантов делегирования.

Прежде чем углубляться в формализацию, зафиксируем *проверяемый* контекст по документации Fuchsia: зачем инварианты capability-модели Zircon интересны инженеру и где типичная точка напряжения — MMIO и слои совместимости.

2. Контекст: актуальность формальной проверки capability-модели

2.1. Индустриальный фон

Fuchsia — операционная система с микроядром Zircon; доступ к объектам ядра строится на capability-дисциплине: процесс работает с ресурсом только через handle с явной маской прав [6, 5]. Эта дисциплина заявлена как основа безопасности и модульности системы. Вопрос настоящей работы не в том, «хороша ли идея в принципе», а в том, можно ли *машинно проверить* узкий, но критичный инвариант делегирования: передача не увеличивает множество прав, — без полной семантики ядра.

2.2. Архитектурный конфликт и место формальной модели

Типичная точка напряжения — драйверы периферии, обращающиеся к регистрам через memory-mapped I/O (MMIO). В Linux такой драйвер часто живёт в привилегированном режиме. В Fuchsia произвольная запись userspace в физическую память *не* свободна: доступ к MMIO-диапазонам выдаётся через ресурсы вида `ZX_RSRC_KIND_MMIO` и связанные механизмы [7]. Параллельно развивается Starnix — слой, эмулирующий Linux-подобные системные вызовы поверх Zircon [8]; маршрутизация capabilities к контейнеру и к `starnix_kernel` там тоже часть дисциплины прав.

Тем не менее исключение вида «открыть чрезмерно широкий MMIO» без узкой маски разрушает смысл capability: процесс с широким доступом к железу снова может записать «что угодно куда угодно», и дефект драйвера превращается в угрозу целостности ядра — ровно то, от чего микроядерная модель должна защищать. Переписывание драйверов и интерфейсов под нативную модель Zircon дорого (IPC, FIDL, границы устройств); без формального языка инвариантов доверие к делегированию прав остаётся декларативным.

Модульные тесты и фаззинг находят конкретные дефекты и не устанавливают свойство вида «ни при каком допустимом состоянии передача handle не увеличивает множество прав». Специалисту по безопасности нужен язык спецификации и способ *опровергнуть*

небезопасную архитектуру ещё до внедрения в код.

Настоящая заметка — методический мост между документацией Zircon [6, 5] и машинно проверяемым фрагментом логики прав: handles/каналы/VMO, отсутствие эскалации, шлюз TRANSFER, целостность записи, решётка, экстракция в OCaml. Цепочка — «архитектура → доказательство → исполняемый артефакт»; модель учебная и не покрывает полный syscall surface.

2.3. Ценность для специалистов по безопасности

Для аудитории, работающей в области безопасности приложений и операционных систем (*application / OS security*), эта работа полезна не как замена полного аудита Zircon, а как *инструментальный образец*.

Язык спецификации
Требования к capability-фрагменту формулируются на языке, который понимает доказатель; неоднозначность проектной документации снижается.

Раннее обнаружение дефектов
Если инвариант не доказывается, архитектура отвергается на этапе модели, а не после инцидента (пример с пропущенным TRANSFER — в §7.1).

Перепроверяемые гарантии
Результат — не только экспертное мнение, а артефакт, который можно пересобрать (`isabelle build`, `test_code`, OCaml-прогон).

Обучение и практики
Компактный пример показывает минимальный набор свойств, которые имеет смысл требовать от capability-IPC до масштабирования на полное ядро.

Ограничение остаётся явным: модель учебная и не покрывает драйверы, Starnix и полный syscall surface; MMIO входит лишь как *контрпример обхода* (§9), а не как полная модель железа. Тем не менее даже такой фрагмент даёт содержательную информацию: инвариант ”нет эскалации” может быть доказан *локально*, без моделирования всей ОС. Ценность для security-инженера — в достижимости формального метода для критичного узла и в шаблоне расширения к тем местам, где безопасность сталкивается с совместимостью.

3. Абстракция: от полного ядра к модели прав

Полная семантика Zircon включает процессы, обширный интерфейс системных вызовов и средства синхронизации. Для анализа логики прав достаточно выделить минимальный набор сущностей и операций.

3.1. Онтология

1. **Идентификаторы.** `process_id`, `handle_id`, `obj_id` моделируются типом `nat`. Выбор отражает интерес к логическим свойствам, а не к битовому представлению 32-разрядных дескрипторов. Очереди каналов индексируются `obj_id` объекта типа `Channel`.

2. **Объект ядра.** Пара $\langle$`тип`, `id`$\rangle$, где тип принадлежит конечному перечислению (`Process`, `Thread`, `Channel`, `VMO`, а также `Hardware` и `Other_Obj` — задел расширяемости; poke в §9 бьёт в `vmo_store` по `obj_id` и тип объекта не проверяет).

3. **Handle** (дескриптор). Ссылка на объект вместе с множеством атомарных прав (capability-маска, rights mask): `record` с полями `h_obj`, `h_rights`. Именно слот `handle` в таблице процесса несёт право доступа; передача `handle` и есть передача полномочия.

4. **Capability** (полномочие). В модели это *тот же тип*: `type_synonym capability = handle`. Имя `capability` подчёркивает роль «в полёте» по каналу (без локального `handle_id` получателя — он появится при `read`); проекция `h_rights` поэтому законна и для элементов `capability list`. Отметим: склейка `capability` и `handle` в один тип — учебное упрощение представления. В реальном Fuchsia права тоже ассоциированы с *handle* (*«Rights are associated with handles»* [5]), а не с записью объекта в таблице ядра; несколько дескрипторов на один объект могут нести разные маски. Наша склейка удобна для модели, но не воспроизводит полное разделение «слот в таблице процесса» / «объект ядра».

5. **Состояние системы** включает:

   a) `proc_tables` — таблицы дескрипторов процессов;

   b) `channels` — очереди capabilities по концам каналов (`obj_id` $\rightharpoonup$ `capability list`);

   c) `channel_peer` — связь конца с противоположным (`obj_id` $\rightharpoonup$ `obj_id`);

   d) `vmo_store` — абстрактное содержимое VMO (`obj_id` $\rightharpoonup$ `nat`);

   e) счётчики для выделения свежих идентификаторов (`next_hids`, `next_obj_id`). Свежие `handle_id` выделяет `alloc_hid pid s`, возвращая пару (новый id, обновлённое состояние со сдвинутым счётчиком); нумерация с 1, поскольку в Zircon `0 = ZX_HANDLE_INVALID`.

Частичная функция. Запись $\alpha \rightharpoonup \beta$ — стандартное для Isabelle обозначение *частичной функции* из α в β: она либо возвращает `Some` y, либо `None`, если значения нет. В терминах теории типов это $\alpha \Rightarrow \beta$ `option`. Такая форма удобна, когда таблица «разрежена»: у не всех `handle_id` есть дескриптор. Именно поэтому *lookup_handle* и *get_table* возвращают `option`, а не отдельный `bool`-признак вместе со значением: отсутствие дескриптора и «дескриптор с пустой маской» — разные состояния.

Короткое пояснение к флагам. Перечисленные в модели права покрывают минимум, необходимый для сценариев статьи. `ZX_RIGHT_READ` и `ZX_RIGHT_WRITE` разрешают соответственно читать и модифицировать объект. `ZX_RIGHT_EXECUTE` разрешает исполнение кода из объекта (актуально для VMO). `ZX_RIGHT_MAP` разрешает отображать объект в адресное пространство процесса. `ZX_RIGHT_TRANSFER` разрешает передавать handle другим процессам через канал — именно это право проверяет *zx_channel_write*. `ZX_RIGHT_DUPLICATE` обязателен для *zx_handle_duplicate*. `ZX_RIGHT_SIGNAL` в текущих сценариях не задействован и приведён для полноты перечня относительно настоящего Zircon (где есть также WAIT, DESTROY, property/policy rights).

Поле `proc_tables` — это частичная функция, принимающая на вход пару (идентификатор процесса, идентификатор дескриптора) и возвращающая полномочие — пару «объект ядра + маска прав». Если для данной пары слот не заполнен, функция возвращает «пусто» (`None`); при этом отсутствие дескриптора и «дескриптор с пустой маской» — два разных состояния, и мы их не смешиваем.

Каждый системный вызов — это атомарная транзакция над `system_state`: таблицы дескрипторов (`proc_tables`) часто в центре внимания, но тот же шаг может менять и другие поля (например, очереди `channels`). Либо состояние целиком переходит в новое (`Some` s'), либо остаётся неизменным (`None`). Промежуточного, частично обновлённого состояния не бывает. Модель последовательная: переходы — атомарные шаги. Это не только упрощение доказательства, но и содержательное допущение: локальные инварианты capability доказуемы именно потому, что шаг неделим. Конкурентность — одновременное обращение нескольких потоков к таблицам дескрипторов — сознательно вынесена за рамки модели.

Отдельная оговорка о времени жизни объектов. В промышленном Zircon на один объект ядра может ссылаться несколько дескрипторов — в одной таблице или в разных; в модели это тоже есть (*zx_handle_duplicate* создаёт второй handle на тот же объект). Чего в модели нет — подсчёта ссылок (*refcounting*): у объекта Zircon есть счётчик, и объект уничтожается, когда закрыт последний handle. У нас *zx_handle_close* удаляет слот, но объект и его запись в `vmo_store` остаются. Добавление подсчёта ссылок — это расширение инварианта корректности состояния (`state_wf`, определён ниже), а не переопределение самих операций.

Наконец, политика порождения новых прав (*minting*) при создании объектов (`create_vmo`, `create_channel_pair`) тоже остаётся вне модели: начальные маски задаются снаружи и не выводятся из прав «родителя». Модель доказывает ослабление прав при передаче и при дублировании дескриптора (*zx_handle_duplicate*), а не корректность того, *кто* вправе породить объект с данной маской.

Канал как capability-объект. Очередь канала адресуется через handle на endpoint — ту же дисциплину, что и для VMO. Ключ в `channels` — `obj_id` объекта типа `Channel`; писать и читать очередь можно только после `lookup_handle` с проверкой `ZX_RIGHT_WRITE` / `ZX_RIGHT_READ`:

```
channels :: "obj_id ⇀ capability list"

zx_channel_write ::
  "process_id ⇒ handle_id ⇒ handle_id ⇒ rights ⇒
   system_state ⇒ system_state option"
  (* pid, channel endpoint, payload, requested rights *)

vmo_write ::
  "process_id ⇒ handle_id ⇒ nat ⇒
   system_state ⇒ system_state option"
```

Пара эндпоинтов создаётся *create_channel_pair*: два объекта `Channel` со своими очередями и взаимной связью `channel_peer`. Запись кладёт сообщение в очередь *пира*, чтение забирает из *своей* — как в Zircon. Оговорка: настоящий `zx_channel_create` возвращает *оба* конца создателю, с одинаковыми правами по умолчанию (`TRANSFER|READ|WRITE|WAIT|INSPECT|SIGNA`), второй конец попадает в другой процесс через bootstrap-handle при `zx_process_start` или по уже существующему каналу. Наша операция склеивает «создать + доставить» в один шаг и позволяет задать маски концов независимо — это учебное упрощение, а не отдельный syscall. Рис. 1 показывает единый путь доступа через handle.

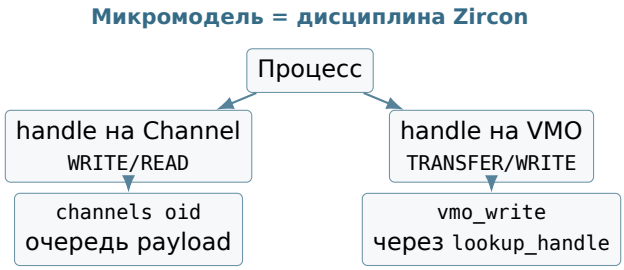


Рис. 1: Единая capability-адресация: и канал, и VMO доступны только через handle в `proc_tables`.

Дополнительно *zx_channel_write* реализует *move*: после успеха `lookup_handle pid pay_hid s' = None` (*channel_write_moves_sender_handle*). Инварианты no-escalation относятся к маске прав у payload. Отношение к промышленному Zircon: обычный `zx_channel_write`

маску не принимает, но `zx_channel_write_etc` принимает — для каждого handle задаётся `zx_handle_disposition_t` с операцией (`MOVE`/`DUPLICATE`) и желаемыми итоговыми правами; запрос сверх прав источника — `ZX_ERR_INVALID_ARGS`, попытка отправить канал в самого себя — `ZX_ERR_NOT_SUPPORTED`. Наша операция — *прообраз* `write_etc` с одной `MOVE`-записью: проверка *req* $\subseteq$ h_rights, затем *reduce_rights_wf* (учебное замыкание зависимостей: итог может быть шире `req`, если замыкание дописывает `READ`), плюс `WRITE` на канал и `TRANSFER` на payload [5, 6]. Как и в `write_etc`, `TRANSFER` в запросе можно опустить — тогда получатель не сможет переслать handle дальше.

Инвариант состояния и трассы. Помимо полей записи важен предикат корректности масок *state_wf*:

```
definition handles_wf where
  "handles_wf s ⟷ ∀(pid ht hid h.
     proc_tables s pid = Some ht ⟶ ht hid = Some h ⟶
     rights_wf (h_rights h))"

definition channels_caps_wf where
  "channels_caps_wf s ⟷ ∀(oid q cap.
     channels s oid = Some q ⟶ cap ∈ set q ⟶
     rights_wf (h_rights cap))"

definition peers_wf where
  "peers_wf s ⟷ ∀(a b. channel_peer s a = Some b ⟶
     channel_peer s b = Some a ∧
     channels s a ≠ None ∧ channels s b ≠ None ∧
     a < next_obj_id s ∧ b < next_obj_id s)"

definition state_wf where
  "state_wf s ⟷ handles_wf s ∧ channels_caps_wf s ∧ peers_wf s"
```

Первые две части говорят об *отдельных* масках: каждая маска в таблицах дескрипторов и в очередях каналов внутренне согласована, то есть удовлетворяет *rights_wf* (наличие `WRITE` или `EXECUTE` требует `READ`). Третья, `peers_wf`, — о топологии каналов: связь концов симметрична, у обоих концов есть очереди, и все упомянутые идентификаторы уже выделены (это нужно, чтобы свежая пара endpoint'ов не могла перезаписать чужую связь). Содержательно это Zircon-утверждение «сообщение попадает к противоположному концу, и у него есть очередь» — доказанное как инвариант трассы, а не только обеспеченное конструкцией *create_channel_pair*.

Важно, чем `state_wf` *не* является: это не отсутствие эскалации из §5. Там речь идёт о *паре* дескрипторов: `h_src` — исходный у отправителя, `h_dst` — приобретённый получателем; теорема *channel_transfer_no_escalation* утверждает $h_rights\ h_dst \subseteq h_rights\ h_src$. Это свойство *шага*, а `state_wf` — предикат на состоянии.

Но эти два уровня связаны, и связь принципиальна для модели. Системные вызовы *не проверяют rights_wf* у источника (как и ядро Zircon при передаче); пошаговая no-escalation берёт её как посылку, а трассовая — извлекает из `state_wf` достижимого состояния. Без посылки утверждение ложно: источник {`WRITE`,`TRANSFER`}, запрос {`WRITE`} — замыкание добавило бы получателю `READ` (*transfer_without_wf_source_may_escalate*). Откуда посылка: создающие операции отклоняют некорректные начальные маски, передающие кладут в таблицы и очереди только замкнутые — на любом достижимом состоянии все маски корректны. Разделение ролей: `state_wf` — инвариант состояния; no-escalation — свойство шага, чья посылка на трассах закрывается инвариантом. Алгебраическая сторона замыкания — в §8.5.1; дальше по тексту — только перекрёстные ссылки сюда, без полного повтора.

В `Zircon_Traces.thy` syscall свёрнуты в метки перехода `datatype step` (конструкторы `Channel_Write`, `Transfer`, `Vmo_Write`, `Duplicate`, `Close`, …) — параметр `st` в правиле достижимости есть выбранная метка. Исполнение одного шага: `exec_step st s = Some s'` либо `None`. Достижимость — рефлексивно-транзитивное замыкание успешных шагов:

```
inductive reachable :: "system_state ⇒ system_state ⇒ bool" where
  refl: "reachable s s"
| step: "reachable s s1 ⟹ exec_step st s1 = Some s2
         ⟹ reachable s s2"
```

Теорема *reachable_preserves_wf*: если `state_wf` s и `reachable` s s', то `state_wf` s'. Security на трассах собрана в *reachable_transfer_no_escalation* и *reachable_duplicate_no_escalation* (`Zircon_Traces.thy`): на каждом успешном `Transfer` / `Duplicate` из reachable-состояния права получателя не превосходят права источника. Доказательство извлекает *rights_wf* источника из `state_wf` достижимого состояния и передаёт её пошаговой лемме.

3.2. Сокращение прав: определение и передача

Центральное проектное решение модели — *определение* (`definition`, не `axiomatization`) операции сокращения прав как пересечения множеств:

```
reduce_rights(new_rights, old_rights) = new_rights ∩ old_rights
```

Тождество с встречей решётки (`inf`) — следствие этого определения и законов на `'a set`; отдельно «доказывать пересечение» не нужно.

Что такое «сырое» пересечение. Под «сырым» пересечением мы понимаем буквально $R_{\text{new}} \cap R_{\text{old}}$ — то, что возвращает *reduce_rights* — *без* последующего замыкания зависимостей. Результат сырого пересечения может нарушать *rights_wf*; поэтому syscall-путь использует замкнутое *reduce_rights_wf*, чей результат *всегда* корректен (*close_rights_wf*). Отдельно от этого стоит *предпосылка rights_wf* у источника: она нужна не для «сломанной маски в очереди» (замыкание её и так починит), а чтобы замыкание не добавило получателю право, отсутствовавшее у источника (канон и контрпример — абзац выше; алгебра — §8.5.1).

Пустая маска и `TRANSFER`. Важно не смешивать два разных факта. *Пустая маска* — допустимое *состояние* handle и допустимый *результат* сокращения (*req*∩*src* $= \emptyset$): в очередь уходит capability с $\bot = \emptyset$ (владение объектом без READ/WRITE), транзакция при этом не обрывается. Но *передача* по-прежнему требует `ZX_RIGHT_TRANSFER` у *исходного* payload-handle: без него *zx_channel_write* возвращает `None`. «Пустой результат пересечения» не означает «можно передать handle без `TRANSFER`». Это разделяет леммы *example_empty_rights_ok_true* (запрос $\{\}$, источник с `TRANSFER`) и *channel_write_requires_transfer*.

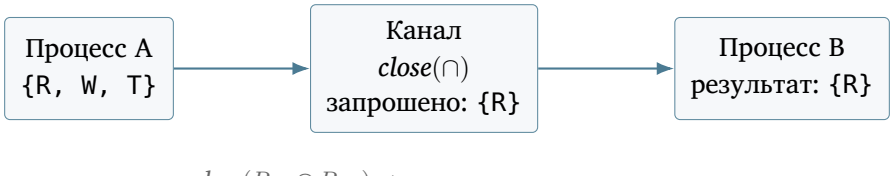


Рис. 2: Схема передачи capability: права получателя равны замкнутому пересечению запрошенного и исходного множеств (*reduce_rights_wf*).

4. Формализация системных операций

Определения Isabelle/HOL синтаксически близки к языкам вроде Haskell или OCaml. Ниже — операция помещения capability в очередь канала *zx_channel_write*; для record-литерала использовано ASCII-представление <|...|> вместо фирменных Isabelle-скобок, которые в `lstlisting` нестабильно рендерятся.

```
definition zx_channel_write ::
  "process_id ⇒ handle_id ⇒ handle_id ⇒ rights ⇒
   system_state ⇒ system_state option" where
  "zx_channel_write pid ch_hid pay_hid req_rights s =
    (case (lookup_handle pid ch_hid s,
           lookup_handle pid pay_hid s, get_table pid s) of
      (Some h_ch, Some h_pay, Some ht) ⇒
        if ch_hid = pay_hid then None
        else if obj_type (h_obj h_ch) ≠ Channel then None
        else if ZX_RIGHT_WRITE ∉ h_rights h_ch then None
        else if ZX_RIGHT_TRANSFER ∉ h_rights h_pay then None
        else if ¬ (req_rights ⊆ h_rights h_pay) then None
        else
          (case channel_peer s (obj_id (h_obj h_ch)) of
            None ⇒ None
          | Some peer ⇒
              (case channels s peer of
                None ⇒ None
              | Some q ⇒
                  (let actual = reduce_rights_wf req_rights (h_rights h_pay);
                       cap = <| h_obj = h_obj h_pay, h_rights = actual |>;
                       ht' = ht(pay_hid := None)
                   in Some (... enqueue on peer ...))))
    | _ ⇒ None)"
```

Разберём определение по существу. Семантика системного вызова здесь задаётся не одной строкой, а совместным действием нескольких особенностей; каждая важна для последующих доказательств.

1. Результат имеет тип `option`: успешное завершение (`Some` s') либо отказ без изменения состояния (`None`). Тем самым исключается частично обновлённое состояние ядра. Это *учебный* выбор, а не семантика ядра: у `zx_channel_write` передаваемые дескрипторы уничтожаются *и при ошибке*; у `write_etc` с `ZX_HANDLE_OP_MOVE` источник тоже закрывается («always closed», как `zx_handle_replace` затем write). У нас `None` означает «состояние не тронуто» — так проще доказывать локальные инварианты.

2. Нужен handle на `Channel` с `ZX_RIGHT_WRITE` и payload-handle с `ZX_RIGHT_TRANSFER`; запрос — *req* $\subseteq h_rights$ (иначе `None`, как `ZX_ERR_INVALID_ARGS` у `write_etc`). Корректность маски payload (*rights_wf*) syscall *не* проверяет — она инвариант состояния (§3.1).

3. Сообщение уходит в очередь *пира* (`channel_peer`); права в нём — *reduce_rights_wf*. Пустой запрос допускается.

4. Исходный payload-handle отправителя *удаляется* (move). Это фиксируется леммой *channel_write_m* после успеха `lookup_handle pid pay_hid s' = None`.

Указанная дисциплина соответствует требованию атомарности (транзакционности) успешного либо полностью отвергнутого перехода состояния — свойству, которое трудно установить исключительно эмпирическими тестами.

4.1. Пошаговый разбор: одна передача

Чтобы читатель, не знакомый с capability-моделями, мог «прочувствовать» семантику *zx_channel_write*, разберём одну конкретную транзакцию пошагово. Это *иллюстративный* микропример (идентификаторы канала 7 и VMO 3 выбраны для наглядности); полный учебный сценарий §6 использует другие id, но ту же механику.

Начальное состояние. Пусть `s` — состояние, в котором таблица процесса 0 содержит два дескриптора:

- `ch0` — handle на канал $\langle$`Channel`, 7$\rangle$ с маской `{READ, WRITE}`;
- `pay` — handle на VMO $\langle$`VMO`, 3$\rangle$ с маской `{READ, WRITE, TRANSFER}`.

У пира конца 7 очередь в `channels` существует (для наглядности микропримера — пуста; на успех write влияет наличие очереди пира, а не её пустота). Процесс 0 вызывает *zx_channel_write* с аргументами `ch_hid = ch0`, `pay_hid = pay`, `req_rights = {READ}`. Разберём, что происходит внутри определения.

Шаг 1. Обратиться к состоянию. Транзакция начинается с трёх обращений: `lookup_handle 0 ch0 s`, `lookup_handle 0 pay s`, `get_table 0 s`. В нашем случае все три возвращают `Some`: оба дескриптора существуют, таблица процесса 0 есть. Если бы хотя бы один вернул `None`, результат был бы `None` без какого-либо изменения состояния — транзакция отвергнута целиком. Это первое проявление атомарности: «не нашли — не начали».

Шаг 2. Проверить условия-стражи. Дальше последовательно проверяются условия:

1. `ch_hid` $\neq$ `pay_hid` — это два разных дескриптора;
2. `obj_type (h_obj ch0) = Channel` — `ch0` смотрит именно на канал;
3. `ZX_RIGHT_WRITE` $\in$ `h_rights ch0` — у отправителя есть право писать в канал;
4. `ZX_RIGHT_TRANSFER` $\in$ `h_rights pay` — у payload-handle есть право быть переданным;
5. *req* $\subseteq$ h_rights *pay* — запрос не шире исходных прав (иначе `None`).

Первые четыре проверяют идентичность слотов, тип и права доступа — в Zircon это `ZX_ERR_NOT_SUPPOR` `ZX_ERR_WRONG_TYPE` и `ZX_ERR_ACCESS_DENIED`. Пятое — запрет тихого урезания сверхзапроса через □: `ZX_ERR_INVALID_ARGS` у `write_etc`. Затем две структурные проверки: у конца есть пир (иначе `ZX_ERR_PEER_CLOSED`) и у пира есть очередь. Чего в списке *нет* — проверки *rights_wf* у источника: syscall ей не занимается, её гарантирует `state_wf`.

Шаг 3. Вычислить маску сообщения. Права, попадающие в очередь, — это не `req_rights` и не `h_rights pay`, а их *замкнутое пересечение*:

$$\begin{aligned} \mathit{actual} &= \mathit{reduce_rights_wf}\ \{\mathrm{READ}\}\ \{\mathrm{READ}, \mathrm{WRITE}, \mathrm{TRANSFER}\} \\ &= \mathit{close_rights}\ \big(\{\mathrm{READ}\} \cap \{\mathrm{READ}, \mathrm{WRITE}, \mathrm{TRANSFER}\}\big) \\ &= \mathit{close_rights}\ \{\mathrm{READ}\} \\ &= \{\mathrm{READ}\}. \end{aligned}$$

Замыкание не сработало: `READ` уже есть, добавлять нечего. Если бы запрос был `{WRITE}`, замыкание добавило бы `READ`, и результат был бы `{READ, WRITE}` — это *не* эскалация, потому что `READ` уже присутствовал в источнике (*reduce_rights_wf_le_old*).

Шаг 4. Мутировать состояние: два действия, одна транзакция. Выполняются ровно две записи в `system_state`:

- `ht(pay := None)` с последующим `set_table 0 ht s` — удаляется payload-handle отправителя (*move*);
- `(channels s)(peer ↦ [⟨VMO 3, {READ}⟩])` — в очередь *противоположного* конца добавляется capability с урезанной маской.

Результат — `Some s'`, где `s'` содержит обе мутации одновременно. Промежуточного состояния, в котором handle уже удалён, а в очередь ещё ничего не положено (или наоборот), не существует — это и есть вторая половина атомарности: «прошли условия-стражи — сделали всё».

Что было бы, если убрать условие-страж. Полезно посмотреть, что сломается при удалении любой из пяти проверок (и что — при ослаблении инварианта состояния). Это то самое «раннее обнаружение архитектурного дефекта» из §7.1 — не абстракция, а конкретный отказ доказательства.

Убранное условие-страж	**Что ломается**
`ch_hid` $\neq$ `pay_hid`	канал отправляется в самого себя (в Zircon — `ZX_ERR_NOT_SUPPORTED`). Move удаляет слот, на который сам же ссылался; семантика *move* теряет смысл, хотя лемма *channel_write_moves_sender_handle* остаётся формально истинной.
тип `Channel` у `ch_hid`	отказ становится *случайным*: транзакция «проходит» или «не проходит» в зависимости от того, есть ли у данного `obj_id` запись в `channels`. Лемма *channel_write_requires_channel_write_right* теряет структурную опору.
`WRITE` у канала	процесс без права писать в канал сможет положить туда сообщение — прямое нарушение capability-дисциплины. Лемма *channel_write_requires_channel_write_right* перестаёт быть доказуемой.
`TRANSFER` у payload	любой handle можно будет передать через канал, включая тот, который по политике для передачи не предназначен. Именно это условие-страж фиксирует *channel_write_requires_transfer*.
req $\subseteq h_rights$	сверхзапрос молча урезается через $\cap$; лемма *channel_write_fails_unless_subset* недоказуема. No-escalation при этом *не* ломается (пересечение не добавляет прав), но теряется соответствие `ZX_ERR_INVALID_ARGS`: вызывающий не узнаёт, что получил меньше, чем просил.
инвариант `rights_wf` у источника (не страж, а часть `state_wf`)	ломается отсутствие эскалации: источник `{WRITE, TRANSFER}`, запрос `{WRITE}` даёт результат `{READ, WRITE}` — получатель получает `READ`, которого у источника не было (*transfer_without_wf_source_may_escalate*). Такое состояние недостижимо: *create_vmo* отклоняет некорректную начальную маску, а все операции сохраняют `state_wf`.

Таблица 1: Каждое условие-страж *zx_channel_write* «работает» на свою лемму; последняя строка — инвариант состояния, а не проверка в syscall. Убрать условие-страж — значит потерять лемму.

Сводка транзакции. Одна передача — это:

- **три обращения** к состоянию (`lookup_handle` $\times 2$ + `get_table` $\times 1$);
- **пять условий-стражей** (разные handle; тип канала; `WRITE`; `TRANSFER`; *req* $\subseteq h_rights$) плюс две структурные проверки (пир существует; у пира есть очередь);
- **одна вычислительная операция** над правами (*reduce_rights_wf* = пересечение + замыкание);
- **две мутации** (`set_table` — удаление слота; `channels` — постановка в очередь);
- **два возможных исхода**: `Some s'` при успехе, `None` без изменения состояния при провале любого условия-стража.

Ни одна мутация не происходит до того, как пройдены все пять условий-стражей; ни одна не пропускается после успеха. Именно эту дисциплину мы имели в виду под «атомарной транзакцией» в предыдущем абзаце. Конкретные гарантии, которые она даёт (необходимость `TRANSFER`, отсутствие эскалации, *move* у отправителя), формулируются как леммы в §5.

5. Доказанные свойства

Утверждения формулируются в виде лемм и теорем; корректность доказательств проверяется Isabelle.

5.1. Необходимость права TRANSFER

```
lemma channel_write_requires_transfer:
  assumes "zx_channel_write pid ch_hid pay_hid req_rights s = Some s'"
  shows "∃h. lookup_handle pid pay_hid s = Some h ∧
             ZX_RIGHT_TRANSFER ∈ h_rights h"
```

Лемма устанавливает машинно проверяемый инвариант: успешный вызов *zx_channel_write* возможен лишь при наличии `TRANSFER` у payload-handle. Симметрично доказывается необходимость `WRITE` у endpoint-handle (*channel_write_requires_channel_write_right*).

5.2. Отсутствие эскалации прав

Теорема утверждает: после успешной передачи через канал у получателя не существует права, отсутствовавшего у отправителя. В `Zircon.thy` она выводится из *transfer_via_channel_no_escal* тактикой `by blast` — по сути это переформулировка «$h_rights\ h_dst \subseteq h_rights\ h_src$» в виде «не существует права у получателя вне источника».

Содержательная работа сделана в *transfer_via_channel_no_escalation*: из успешной передачи извлекается факт $h_rights\ h_dst = reduce_rights_wf\ req\ h_rights\ h_src$ (лемма *transfer_via_channe* и применяется *reduce_rights_wf_le_old*, которое утверждает вложение

$$reduce_rights_wf\ req\ src \subseteq src \quad \text{при условии} \quad rights_wf\ src.$$

Оговорка про *rights_wf* существенна: без неё лемма неверна (замыкание могло бы добавить `READ`). Syscall её не проверяет; посылка берётся из `state_wf`, а на трассах — извлекается в *reachable_transfer_no_escalation* (§13.4). Развёрнутое разъяснение и контрпример *transfer_without_wf_source_may_escalate* — в §3.1; алгебраическая сторона — в §8.5.1.

Итоговая цепочка

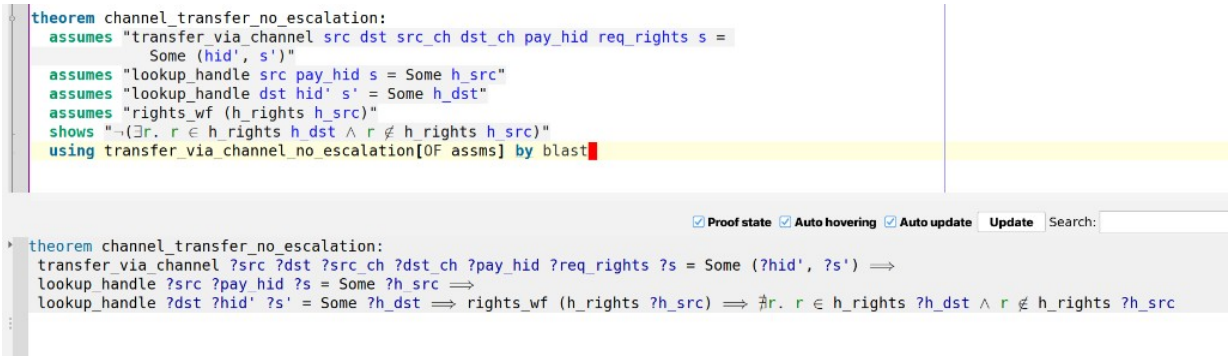


Рис. 3: Isabelle/jEdit (PIDE): формулировка *channel_transfer_no_escalation* в буфере `Zircon.thy` и принятое утверждение в панели Output. Доказательство — `by blast` из *transfer_via_channel_no_escalation*; сокращение прав — замкнутое пересечение (*reduce_rights_wf*). Предпосылка *rights_wf* у источника на снимке явная; в трассовых теоремах она выводится из `state_wf`, а не из стража syscall'а.

$$\underbrace{h_rights\ h_dst}_{\text{права получателя}} = \mathit{reduce_rights_wf}\ \mathit{req}\ h_src \subseteq \underbrace{h_rights\ h_src}_{\text{права источника}}$$

и есть то, что читает *channel_transfer_no_escalation*. Лемма *transfer_rights_eq_meet* из `Zircon_Lattice.thy` устанавливает то же равенство на языке решёток (имя *meet* — отсылка к встрече; строгое *reduce_rights* = `inf` — в §8.1), но *не* участвует в доказательстве теоремы ядра: теория решётки импортирует `Zircon`, а не наоборот. Решёточные варианты — параллельная иллюстрация, а не основание главных security-лемм.

На рис. 3 видно, как PIDE отображает формулировку в редакторе и в Output после успешной проверки. В отличие от единичного тестового примера, утверждение относится ко всем состояниям, удовлетворяющим предпосылкам модели. Конкретная транзакция, разобранная в §4 (*Пошаговый разбор*), и таблица 1 показывают, что у каждого из пяти условий-стражей есть *своя* лемма (или свой аспект семантики): убрать страж — значит потерять именно её, а не обязательно *channel_transfer_no_escalation*. Например, без *req* $\subseteq h_rights$ no-escalation сохраняется (пересечение не добавляет прав), но ломается соответствие `ZX_ERR_INVALID_ARGS`; без `ch_hid` $\neq$ `pay_hid` лемма о move остаётся формально истинной, хотя смысл self-move теряется. Успех самой *channel_transfer_no_escalation* наследует ещё и ограничение композиции *transfer_via_channel*: очередь получателя должна быть пуста (см. границу модели); у *zx_channel_write* пустота не требуется.

5.3. Дублирование дескриптора: тот же meet

Сокращение прав — это не только операция передачи по каналу. Тот же приём используется в модели ещё раз, при дублировании дескриптора: операция *zx_handle_duplicate* создаёт *новый* дескриптор на тот же объект ядра, но с урезанной маской. В реальном Zircon такой вызов используется постоянно — например, когда один сервис хочет дать другому только часть своих полномочий на объект, не теряя собственный доступ. Свежий идентификатор дескриптора выделяет `alloc_hid`; исходный дескриптор при этом *сохраняется*, то есть операция работает как копирование с ослаблением прав, а не как передача:

```
definition zx_handle_duplicate ::
  "process_id ⇒ handle_id ⇒ rights ⇒ system_state ⇒
   (handle_id × system_state) option" where
  "zx_handle_duplicate pid hid req_rights s =
    (case (lookup_handle pid hid s, get_table pid s) of
      (Some h, Some ht) ⇒
```

```
        if ZX_RIGHT_DUPLICATE ∉ h_rights h then None
        else if ¬ (req_rights ⊆ h_rights h) then None
        else
          (let actual = reduce_rights_wf req_rights (h_rights h);
               h' = h <| h_rights := actual |>;
               (hid', s1) = alloc_hid pid s
           in Some (hid', set_table pid (ht(hid' ↦ h')) s1))
    | _ ⇒ None)"
```

Лемма no-escalation для duplicate выписывается явно (тот же meet + *reduce_rights_wf_le_old*):

```
lemma zx_handle_duplicate_no_escalation:
  assumes "zx_handle_duplicate pid hid req_rights s = Some (hid', s')"
      and "lookup_handle pid hid s = Some h"
      and "lookup_handle pid hid' s' = Some h'"
      and "rights_wf (h_rights h)"
  shows "h_rights h' ≤ h_rights h"
```

Предпосылка *rights_wf* — как у канала (§3.1); трассовая форма — *reachable_duplicate_no_escalation*. Рядом введён *zx_handle_close* (удаление слота): в Zircon семантика `ZX_HANDLE_OP_MOVE` у `zx_channel_write_etc` эквивалентна `zx_handle_replace` с последующей записью; в модели move встроен в *zx_channel_write*, а close оставлен явным (без счётчика ссылок на объект — см. границу модели в начале статьи), чтобы таблица дескрипторов была полнее для будущего refinement. Обе операции есть и как метки трасс (`Duplicate`, `Close`), так что `state_wf` доказан сохраняющимся и через них. Методический вывод: передающие операции (канал, duplicate) — это один и тот же meet, применённый к разным входам.

6. Примерный сценарий

Учебный сценарий задан прямо в теории: в `Zircon.thy` он зафиксирован как замкнутый булев предикат `example_scenario` и *принят* леммой *example_scenario_true* (`"example_scenario`. Isabelle разворачивает определения операций и проверяет конкретную трассу целиком — этого достаточно, чтобы убедиться, что все определения согласованы между собой на одном конкретном входе.

Подчеркнём эпистемический статус этого предиката. Лемма *example_scenario_true* — это *свидетель* (конкретный пример, на котором модель ведёт себя ожидаемо), а не универсальная теорема. Она не заменяет *channel_transfer_no_escalation*: та утверждает свойство для *всех* состояний с данными предпосылками, а сценарий — лишь для одного. Контрапозиция к `example_scenario` была бы контрпримером к *определениям* модели (или к самому сценарию), а не опровержением универсальной теоремы.

По шагам предикат фиксирует:

1. *create_channel_pair* выдаёт процессам 0 и 1 по handle на два конца одного канала (пиры, связанные `channel_peer`);
2. процесс 0 создаёт VMO с содержимым 42 и правами `READ | WRITE | TRANSFER`;
3. дескриптор передаётся процессу 1 с запросом только `READ` (move);
4. у получателя есть `READ`, нет `WRITE`; у отправителя payload-handle исчез;
5. `vmo_write` процесса 1 значения 99 возвращает `None`;
6. содержимое VMO остаётся `Some 42`.

```
lemma example_scenario_true: "example_scenario"
  unfolding example_scenario_def ... by simp
```

Сценарий независим от конкретной аппаратной платформы и компилятора ядра: он фиксирует свойство *модели*, воспроизводимое при пересборке теории. Тот же смысл переносится в исполняемый слой: `Zircon_Code.thy` определяет *example_exec_ok*, доказывает *example_exec_ok_true* и проверяет выражение через `test_code` / OCaml.

7. Методические следствия

7.1. Раннее обнаружение архитектурных дефектов

Ошибка спецификации вида ”передача без проверки `TRANSFER`” в реализационном коде может проявиться лишь в эксплуатации. В формальной модели отсутствие соответствующей проверки препятствует доказательству леммы *channel_write_requires_transfer*: теория не собирается, пока шлюз не восстановлен. Это и есть ранний сигнал ”архитектура небезопасна”, даже если промышленный код ещё не написан.

7.2. Оценка трудозатрат

Для полной учебной Isabelle-сессии этой работы (см. глоссарий; каналы с handles и move, решётка, зависимые права, трассы, MMIO-контрпример, экстракция и `value/eval`) реалистичны следующие порядки:

- в сопровождающем репозитории — порядка 2500 строк по шести теориям сессии (`Portable_Capabil` `Zircon.thy`, `Zircon_Lattice.thy`, `Zircon_Traces.thy`, `Zircon_MMIO.thy`, `Zircon_Code.thy`; `wc -l`, после правок peer/DUPLICATE/subset), из них порядка двух третей — определения и сопровождающий текст, остальное — леммы и доказательства;
- от нескольких дней до пары недель на построение, отладку доказательств и настройку экстракции — в зависимости от опыта с Isabelle;
- Isabelle/HOL как свободно распространяемый инструмент с документацией.

Это всё ещё *на порядки* меньше промышленного порта драйвера или полного syscall surface, но уже не «сотня строк за вечер»: читатель, воспроизводящий модель, должен ожидать объём курса/мини-проекта, а не лабораторной заготовки на час. Формализация отвечает на вопрос об инварианте делегирования прав, который можно зафиксировать локально и относительно дёшево по сравнению с полным портом железа — но не бесплатно.

Масштаб для сравнения. Порядок приведённых цифр стоит сопоставить с масштабом полной верификации: проект seL4 потребовал порядка двух десятков человеко-лет и сотен тысяч строк скриптов Isabelle/HOL [4]. Наша заметка не конкурирует с этим масштабом — она показывает, что проверка *архитектурных* инвариантов делегирования возможна на порядки дешевле и на порядки *раньше*, чем полная верификация. Именно эта «ранняя» роль микромодели нас здесь и интересует: отвергнуть небезопасную архитектуру до того, как в неё вложены человеко-годы.

7.3. Соотношение с тестированием и фаззингом

Формальная проверка инвариантов не заменяет эмпирические методы, а дополняет их иным классом гарантий. Табл. 2 сводит четыре свойства, по которым эти классы различаются.

Таблица 2: Сопоставление методов обеспечения качества для инварианта отсутствия эскалации прав.

Свойство	**Тесты**	**Фаззинг**	**Форм. вер.**
Доказанное отсутствие эскалации	—	—	да
Общность по состояниям модели	—	—	да
Автоматизированная проверка	да	да	да
Быстрое обнаружение частных дефектов	да	да	частично

Два верхних свойства — «доказанное отсутствие эскалации» и «общность по состояниям модели» — принципиально недостижимы ни тестированием, ни фаззингом. Тест проверяет поведение на конкретных входах; он не может перебрать все допустимые состояния и не может установить свойство вида «для всех s». Фаззинг идёт дальше в смысле охвата входов, но сам по себе остаётся эмпирическим: успешный прогон фаззера не доказывает отсутствия дефекта, он лишь не нашёл его за отведённое время.

Автоматизированная проверка у всех трёх методов есть, но её природа различна. У тестов и фаззера это прогон на наборе входов; у формальной верификации — сборка теории Isabelle, где каждая тактика либо завершается успешно, либо падает. Именно последнее свойство делает вклад формального метода уникальным: успешная сборка теории не оставляет возможности «не заметить» контрпример, тогда как успешный тест или фаззер всегда допускает, что нужный вход просто не был предъявлен.

Быстрое обнаружение частных дефектов — сильная сторона тестов и фаззинга и лишь частичная — формальной верификации. Доказательство не заменяет модульный тест: если ошибка лежит *вне* формальной модели (например, в кодогенераторе или в обработке пользовательского ввода), Isabelle её не увидит. Формальная верификация и эмпирические методы — не конкуренты, а разные инструменты одного процесса обеспечения качества.

8. Алгебраическая структура прав: решётка

До сих пор права трактовались как множества, а сокращение — как пересечение. За этой операцией стоит стандартная алгебраическая структура: семейство всех подмножеств атомарных флагов с операциями $\cap$ и $\cup$ образует *дистрибутивную решётку* (distributive lattice).

Стоит сразу оговорить статус равенства $\mathit{reduce_rights} = \texttt{inf}$. Это *наше* определение — технический приём, облегчающий доказательство, а не «открытие о природе Zircon». Сокращение прав можно было бы определить и пересечением с учётом приоритетов, и сужением по маске, и любой другой монотонной операцией; определения системных вызовов остались бы осмысленными. Но именно пересечение даёт самые короткие машинно проверяемые выводы, поскольку опирается на уже встроенную в HOL алгебру на `'a set`. В Isabelle/HOL эта структура доступна для типа `'a set` (следовательно, и для `rights`) через классы `semilattice_inf/semilattice_sup`, `lattice` и `distrib_lattice` теории `Main` (см. также [1, 2]).

Поскольку тип `rights` является синонимом для `set right`, все операции решётки (`inf`, `sup`, $\leq$) доступны автоматически. Отдельный `instantiation` для синонима типа невозможен и не нужен: мы явно фиксируем равенство $\mathit{reduce_rights} = \texttt{inf}$ и выводим свойства модели как следствия законов решёток (при желании тот же смысл можно оформить через `interpretation`, но для данной работы достаточно прямого равенства).

8.1. Права как решётка

Встреча (meet; в Isabelle — `inf`, теоретико-множественно — пересечение $\cap$) на множествах прав совпадает с $\cap$; соединение (join; `sup`, объединение $\cup$) — с $\cup$. Тем самым автоматически выполняются коммутативность, ассоциативность, идемпотентность и законы поглощения. Ниже в формулах для сокращения прав используем именно $\cap$ / `inf` (встречу), а не $\cup$ / `sup` (соединение).

В теории `Zircon_Lattice` это фиксируется равенством

```
lemma reduce_rights_eq_inf:
  "reduce_rights new_rights old_rights = inf new_rights old_rights"
  by (simp add: reduce_rights_def)
```

и следствиями

```
lemma reduce_rights_commute:
  "reduce_rights A B = reduce_rights B A"
  by (simp add: reduce_rights_eq_inf inf_commute)

lemma reduce_rights_assoc_via_lattice:
  "reduce_rights R1 (reduce_rights R2 R) =
     reduce_rights (inf R1 R2) R"
  by (simp add: reduce_rights_eq_inf inf_assoc)
```

Последнее утверждение — ассоциативность цепочки сокращений (ассоциативность встречи), а не идемпотентность; идемпотентность `inf` формулируется отдельно (`reduce_rights A A = A`).

8.2. Частичный порядок

Порядок на правах совпадает с включением: $A \leq B$ означает $A \subseteq B$. Пустое множество — наименьший элемент $\bot$; `UNIV` (полный набор флагов типа `right`) — наибольший элемент $\top$ в данной модели. «Уменьшение прав» при передаче — переход к элементу, меньшему или равному исходному в этом порядке.

```
lemma reduce_rights_le_old:
  "reduce_rights new_rights old_rights ≤ old_rights"
  by (simp add: reduce_rights_eq_inf)

lemma reduce_rights_greatest:
  assumes "C ≤ new_rights" "C ≤ old_rights"
  shows "C ≤ reduce_rights new_rights old_rights"
  using assms by (simp add: reduce_rights_eq_inf)
```

Вторая лемма выражает характеризацию встречи как *наибольшей нижней грани* (greatest lower bound) — то есть как самого «щедрого» из допустимых результатов.

Развернём это человеческим языком. Пусть даны два ограничения: набор прав не должен выходить за запрос и не должен выходить за источник. Среди всех наборов, которые одновременно удовлетворяют обоим ограничениям, существует наибольший — и это в точности результат сокращения. Любой другой набор, подходящий под оба ограничения, вложен в него. Именно поэтому модель не «урезает права сверх необходимого»: она возвращает максимум из допустимого.

8.3. Переформулировка свойств безопасности

Отсутствие эскалации. Закон $\mathit{inf}(a, b) \leq b$ (эквивалентно $a \cap b \subseteq b$) непосредственно даёт: результат сокращения не содержит прав вне исходного набора.

```
lemma no_escalation_via_meet:
  "¬∃(r. r ∈ reduce_rights req src ∧ r ∉ src)"
  using reduce_rights_le_old[of req src]
  by (auto simp: reduce_rights_def)
```

Передача как встреча с замыканием. Для успешного *transfer_via_channel* права получателя в точности равны *встрече запроса и прав отправителя с последующим замыканием зависимостей* — то есть *reduce_rights_wf*, а не сырому `inf`:

```
lemma transfer_rights_eq_meet:
  assumes "transfer_via_channel ... = Some (hid', s')"
      and "lookup_handle src pay_hid s = Some h_src"
      and "lookup_handle dst hid' s' = Some h_dst"
  shows "h_rights h_dst =
           reduce_rights_wf req_rights (h_rights h_src)"
```

Отсюда

```
theorem channel_transfer_no_escalation_via_lattice:
  ...
  shows "h_rights h_dst ≤ h_rights h_src"

theorem channel_transfer_respects_request_via_lattice:
  ...
  shows "h_rights h_dst ≤ close_rights req_rights"
```

то есть одновременно $R_{\text{dst}} \le R_{\text{src}}$ и $R_{\text{dst}} \le \mathit{close}(R_{\text{req}})$ (замыкание запроса — верхняя оценка после *reduce_rights_wf*).

Монотонность по отправителю. Расширение прав отправителя не уменьшает результат при фиксированном запросе (монотонность `inf` по одному аргументу):

```
lemma reduce_rights_mono_old:
  assumes "old_rights ≤ old_rights'"
  shows "reduce_rights req old_rights ≤
           reduce_rights req old_rights'"
  using assms by (auto simp: reduce_rights_eq_inf)
```

8.4. Соединение (join) и агрегация прав

Передача capability опирается на *встречу*. Для сценариев слияния (наследование, объединение пространств имён) нужна двойственная операция соединения:

```
definition combine_rights :: "rights ⇒ rights ⇒ rights" where
  "combine_rights A B = (A ∪ B)"   (* = sup *)

lemma combine_rights_ge_left:  "A ≤ combine_rights A B"
lemma combine_rights_ge_right: "B ≤ combine_rights A B"
lemma combine_may_exceed_left:
  "¬ (combine_rights {} {ZX_RIGHT_READ} ≤ {})"
```

Здесь {} — нижний элемент решётки $\bot$ (пустой набор флагов как элемент порядка), а не «handle с пустой маской в смысле §3.2»; лемма лишь показывает, что join может выйти за левый аргумент. Join — наименьшая верхняя грань, но *не* закон no-escalation относитель-

но одного родителя: результат может превосходить каждый из аргументов. Дистрибутивность решётки даёт

```
lemma reduce_rights_distrib_join:
  "reduce_rights (combine_rights A B) C =
     combine_rights (reduce_rights A C) (reduce_rights B C)"
```

то есть сокращение агрегата — агрегат сокращений.

8.5. Зависимые права

Минимальная учебная зависимость: наличие `WRITE` или `EXECUTE` влечёт `READ`. Это фиксируется определением

```
definition rights_wf :: "rights ⇒ bool" where
  "rights_wf R ≡
     (ZX_RIGHT_WRITE ∈ R ⟶ ZX_RIGHT_READ ∈ R) ∧
     (ZX_RIGHT_EXECUTE ∈ R ⟶ ZX_RIGHT_READ ∈ R)"
```

Предикат *close_rights* замыкает маску по зависимостям (добавляет `READ`, если есть `WRITE`/`EXECUTE`). Само замыкание идемпотентно всюду (`close_rights (close_rights R) = close_rights R`); лемма ниже — иное свойство: она *характеризует корректные маски как неподвижные точки close_rights*:

```
lemma close_rights_idem:
  "close_rights (close_rights R) = close_rights R"

lemma close_rights_fix_wf:
  "rights_wf R ⟷ close_rights R = R"
```

Определения *close_rights*/*rights_wf* и обе леммы живут в `Zircon.thy`; здесь, в решёточном разделе, они *используются* как мост к *reduce_rights_wf*.

Оговорка о реализме. В настоящем Zircon права `ZX_RIGHT_READ` и `ZX_RIGHT_WRITE` *независимы*: handle может позволять запись без чтения, и наоборот. Зависимость `WRITE` ⇒ `READ` введена здесь *искусственно* — как минимальный пример, на котором видно, зачем нужен отдельный шаг замыкания *close_rights* поверх сырого пересечения. Это не разрешение «раз есть `WRITE`, можно выдать и `READ`»: замыкание *копирует* бит, который у корректного источника уже был, и только поэтому не ломает no-escalation. Читатель, работающий с другим ядром, может заменить эту зависимость на любую другую (например, `EXECUTE` ⇒ `MAP` ⇒ `READ`, см. §8.5.1) или на пустое отношение — в последнем случае *close* = *id*, и три аксиомы замыкания тривиальны. Структура доказательств не меняется.

8.5.1. Ключевая лемма: почему замыкание не нарушает no-escalation

Сырое пересечение может нарушить зависимости:

```
lemma reduce_rights_may_break_wf:
  "rights_wf {READ, WRITE} ∧
   ¬ rights_wf (reduce_rights {WRITE} {READ, WRITE})"

definition reduce_rights_wf where
  "reduce_rights_wf req src = close_rights (reduce_rights req src)"
```

close_rights формально *добавляет* `READ` к маске. Казалось бы, это эскалация относительно голого meet. Главная лемма раздела снимает кажущийся парадокс:

```
lemma reduce_rights_wf_le_old:
  assumes "rights_wf src"
  shows "reduce_rights_wf req src ≤ src"
```

Она говорит не «no-escalation через meet» (это следствует из `inf` $\leq$ аргумента), а сильнее по смыслу для модели с зависимостями: *no-escalation несмотря на замыкание*. Добавленные замыканием флаги уже обязаны быть в `src`, иначе источник не удовлетворял бы `rights_wf`. Получатель видит `READ` рядом с `WRITE` не потому, что модель дарит чтение, а потому, что у источника оно уже было.

Разделим два мотива. *Зачем reduce_rights_wf*, а не сырое пересечение: результат замыкания всегда удовлетворяет *rights_wf* (*close_rights_wf*), так что в очередь не попадёт «сломанная» маска. *Зачем* предпосылка *rights_wf* у источника: без неё замыкание может *добавить* `READ`, которого в источнике не было — и сломать no-escalation, хотя маска получателя останется корректной. Для самой леммы *reduce_rights_wf_le_old* в доказательстве достаточно монотонности `close` и одной стороны `close_fix_wf` (*wf* $\Rightarrow$ *close* $=$ *id*); идемпотентность нужна для алгебры замыкания в целом, а не как единственная опора этой леммы. Закрытие — бухгалтерия зависимостей, а не послабление политики. Именно к *reduce_rights_wf_le_old* придётся вернуться, добавляя новые рёбра зависимостей (например, `EXECUTE`⇒`MAP`⇒`READ`). В §13.2 та же лемма появляется как следствие locale `portable_capability`: «парадокс» исчезает, когда замыкание задано аксиомами монотонности, инфляции и неподвижных точек.

Лемма *reduce_rights_may_break_wf* — мотивация введения замыкания, а не «баг модели». В syscall-пути *zx_channel_write* и *zx_handle_duplicate* используют *reduce_rights_wf*; *rights_wf* у источника — из `state_wf`, а не из стража syscall'а (канон — §3.1). Явная проверка *rights_wf* — только в `create_vmo` / `create_channel_pair`.

8.6. Алгебраический разбор сценария

Для $R_{\text{src}} = \{\texttt{READ}, \texttt{WRITE}, \texttt{TRANSFER}\}$ и $R_{\text{req}} = \{\texttt{READ}\}$ результат — *встреча* (meet / `inf` / $\cap$), а не соединение (join / $\cup$):

$$R_{\text{dst}} = R_{\text{src}} \cap R_{\text{req}} = \{\texttt{READ}\}.$$

В этом частном случае замыкание не срабатывает: *reduce_rights_wf* совпадает с *reduce_rights*, потому что {READ} уже содержит READ. Различие с § о зависимых правах видно на запросе {WRITE} против того же источника: сырое пересечение даёт {WRITE}, а *reduce_rights_wf* — {READ,WRITE} (см. также `value/eval` в разделе экстракции). Машинно для «чистой» встречи:

```
lemma example_meet_computation:
  "reduce_rights {ZX_RIGHT_READ}
      {ZX_RIGHT_READ, ZX_RIGHT_WRITE, ZX_RIGHT_TRANSFER}
   = {ZX_RIGHT_READ}"
  by (simp add: reduce_rights_def)
```

Свойство наибольшей нижней грани объясняет, почему модель не «урезает права сверх необходимого»: среди всех наборов, допустимых и запросом, и исходной маской, выбирается наибольший.

8.7. Методический эффект решёточной абстракции

Как видно из табл. 3, решёточный подход меняет не только формулировки, но и способ переиспользования доказательств: вместо изолированных рассуждений о $\cap$ появляются следствия уже встроенной в HOL алгебры.

Таблица 3: Сопоставление формулировок без явной решётки и с опорой на неё.

Аспект	Без решётки	С решёткой
Язык свойств	операции на множествах	законы `inf/sup`/$\leq$
Доказательства	отдельные леммы о $\cap$	следствия классов HOL
Масштабирование	каждое свойство заново	наследование алгебры
Интерпретация передачи	«пересечение не добавляет прав»	«передача — встреча в решётке»
Расширение набора прав	новые флаги требуют ручной проверки всех лемм	новые атомы прав наследуют законы `inf/sup` (гипотетически; в модели их нет)

Аппарат решёток не отменяет конкретные леммы о каналах и VMO, а поднимает их на уровень, где «нет эскалации» читается как частный случай монотонности и свойства нижней грани — хорошо изученных фактов универсальной алгебры, уже встроенных в Isabelle/HOL.

9. Контрпример MMIO: когда шлюз capability недостаточен

Индустриальный конфликт «безопасность vs совместимость» (драйверы модема, Starnix-подобные слои) формализуется не расширением списка «хороших» syscall, а явным *обходом*. Важно: это *гипотетический* немедиируемый poke, а не модель реального пути Fuchsia, где MMIO выдаётся через ресурс `ZX_RSRC_KIND_MMIO` и физический VMO. Теория `Zircon_MMIO.thy` вводит привилегированную запись в `vmo_store` по сырому `obj_id` — без `lookup_handle` и без проверки `WRITE`. В модели поле `vmo_store` имеет тип `obj_id` $\rightharpoonup$ `nat`: содержимое адресуется идентификатором объекта, а не handle; обычный *vmo_write* сначала решает `obj_id` через таблицу дескрипторов, poke — нет (снова `<|...|>` вместо Isabelle record-update):

```
definition privileged_vmo_poke ::
  "obj_id ⇒ nat ⇒ system_state ⇒ system_state" where
  "privileged_vmo_poke oid content s =
     s <| vmo_store := (vmo_store s)(oid ↦ content) |>"
```

В отличие от *vmo_write*, переход никогда не возвращает `None`: нет handle, нет типовой проверки, нет маски прав.

Свидетель *mmio_leak_witness* — булев предикат в том же стиле, что и *example_scenario*: учебный путь (канал, VMO 42, передача только `READ`), отказ `vmo_write` у получателя, затем привилегированный poke, меняющий содержимое на 99 при всё ещё отсутствующем `WRITE`:

```
definition mmio_leak_witness :: bool where
  "mmio_leak_witness =
    (... create_channel_pair; create_vmo 0 ... 42;
         transfer_via_channel ... {ZX_RIGHT_READ} ...;
     case lookup_handle 1 hid_b s3 of
       Some h_b ⇒
         ZX_RIGHT_WRITE ∉ h_rights h_b ∧
         vmo_write 1 hid_b 99 s3 = None ∧
         vmo_content oid s3 = Some 42 ∧
         (let s4 = privileged_vmo_poke oid 99 s3
          in vmo_content oid s4 = Some 99 ∧
             lookup_handle 1 hid_b s4 = Some h_b ∧
```

```
             ZX_RIGHT_WRITE ∉ h_rights h_b)
    | _ ⇒ False)"

lemma mmio_leak_witness_true: "mmio_leak_witness"
```

Лемма лишь принимает этот предикат (`by simp` после разворачивания определений). Архитектурное чтение: леммы capability-IPC при этом *не опровергнуты* — они по-прежнему истинны для опосредованных переходов. Опровергнуто системное утверждение вида «store меняется только через `vmo_write`»: достаточно добавить один неперехватываемый путь, и изоляция по маске перестаёт быть свойством всей системы. Именно такова форма компромисса «открыть MMIO спец-syscall'ом» из индустриального контекста выше.

Разные эпистемологии: witness vs отсутствие обходов. Подход seL4 к обходам — *доказать их отсутствие*: инварианты политики сохраняются на всём стеке refinement, включая обработчики прерываний и управление таблицами страниц [4]. Мы используем двойственный приём: не доказываем отсутствие обходов, а предъявляем конкретный обход как *negative witness* (негативный свидетель, контрпример к утверждению) (*mmio_leak_witness_true*). Разница принципиальна. Доказательство отсутствия обходов требует полной модели железа, обработчиков прерываний и DMA (в Zircon доступ к MMIO выдаётся через ресурс `ZX_RSRC_KIND_MMIO`, а DMA ограничивается IOMMU через bus transaction initiator, BTI) — то есть работы масштаба seL4. Negative witness требует лишь одной строчки, описывающей обход, и одного доказательства, что он ломает конкретный инвариант. Именно поэтому negative witness — реалистичный инструмент *ранней* аттестации архитектуры, тогда как полное доказательство отсутствия обходов — задача уровня сертификации ядра.

10. От модели к коду: проблематика refinement

Доказанные леммы живут в мире *абстрактных* переходов $s \mapsto$ `Some` s'. Читатель, привыкший к промышленным ядрам, закономерно спрашивает: что связывает эту картину с тысячами строк C++ в Zircon, с гонками на таблицах дескрипторов, с оптимизациями компилятора и с драйверами, которые «просто пишут в регистр»? Ответ не в расширении списка syscall на всё ядро — для обзорной статьи это непосильный и методологически чуждый шаг, — а в понятии *уточнения* (refinement): отношения между слоями описания одной и той же системы.

Здесь важно не смешивать два разных желания. Первое — *покрыть больше API*: добавить порты, события, полный набор rights, планировщик. Второе — *спуститься ближе к коду*, который действительно исполняется. Первое увеличивает ширину модели; второе — глубину доверия. Обзорная работа сознательно выбирает глубину: лучше честно объяснить, чего ещё нет между HOL-переходом и машинной инструкцией, чем обещать «ещё немного syscall — и мы у верифицированного ядра».

10.1. Три этажа одной башни

Удобно мыслить три этажа — не как иерархию важности, а как уровни *наблюдаемости* одного и того же замысла.

1. **Политика / модель** (наш Isabelle/HOL). Здесь формулируются объекты, операции и инварианты: no-escalation, move, `rights_wf`, сохранение `state_wf` на трассах, контрпример MMIO. Язык — математика с исполняемыми определениями; ошибки здесь стоят дешево и ловятся доказательством.

2. **Исполняемая спецификация**. Та же семантика, но уже не в виде математических утверждений, а в виде *программы*: её можно запустить и получить результат. На прак-

тике это либо автоматическая экстракция в язык программирования — в нашем случае в OCaml (см. §11), — либо рукописный эталонный модуль на функциональном языке. На этом уровне сценарии можно *вычислять* (`value`, `eval`, `test_code`) и сравнивать результаты с тем, что утверждают леммы. Это ещё не ядро — нет ни планировщика, ни работы с железом, — но и не «чистая бумага»: у кода появляются типы целевого языка, порядок вычислений и границы модулей. Именно здесь модель превращается в артефакт, который инженер может запустить у себя на машине.

3. **Код ядра** (C/C++ Zircon, ассемблерные вставки, DMA — прямой доступ устройств к памяти). Именно он исполняется на железе. Между вторым и третьим этажом пролегает пропасть: указатели, алиасинг (несколько ссылок на одну память), прерывания, модель памяти, *ABI* системных вызовов (двоичный интерфейс, через который пользовательский код вызывает ядро), взаимодействие с драйверами.

Refinement — дисциплина, которая делает эту пропасть *явной* и частично устранимой: каждое поведение нижнего этажа должно быть допустимым поведением верхнего (или наоборот — в зависимости от выбранного направления симуляции). Пока отношение не установлено, фраза «мы доказали безопасность capability-IPC» относится строго к этажу 1. Она не ложна — она *узка*: истинна для модели, молчит о бинарнике.

Можно представить башню иначе: этаж 1 говорит, *что разрешено*; этаж 2 — *как это считается* в идеализированной машине; этаж 3 — *что происходит* на реальном процессоре. Уточнение — договор о том, что нижние этажи не изобретают лишних историй сверх разрешённых верхними.

Refinement как промышленная дисциплина. Классическая схема refinement в seL4 [4] содержит именно эти уровни — абстрактная спецификация, исполняемая monadic-спецификация, C-код и, наконец, бинарник — и доказывает симуляцию между каждой парой соседних уровней. Наша модель остаётся на верхнем этаже: мы формулируем политику, доказываем её свойства и останавливаемся. Ссылка на seL4 здесь нужна не для того, чтобы присвоить себе часть её результата, а чтобы читатель видел: «башня» — не авторская метафора, а редуцированная версия промышленного фреймворка. Уровни, которые мы *не* проходим, перечислены явно; цена их прохождения — тоже.

10.2. Что именно обещает уточнение

Классическая схема (в духе проектов seL4 [3], CertiKOS, частей Verve) звучит так: если реализация C *уточняет* спецификацию S (refinement, доказанная *симуляция*: каждое поведение C допустимо как поведение S), а в S доказан инвариант I, то I переносится на C — при условии, что уточнение сохраняет *наблюдаемое поведение*, интересное для I, то есть все свойства, которые мы умеем проверять на уровне C. Например, наблюдение $h_\textit{rights}(h_{\text{dst}}) \le h_\textit{rights}(h_{\text{src}})$ должно быть выражено через *наблюдаемые* поля состояния в C (биты в таблице дескрипторов ядра), а не через абстрактные `h_rights` модели S. Для нашей модели это означало бы, например:

- успешный `zx_channel_write` в ядре не выдаёт получателю прав сверх пересечения запроса и маски отправителя;
- move действительно удаляет локальный дескриптор отправителя;
- запись в VMO без `WRITE` в опосредованном syscall невозможна.

Без refinement те же утверждения остаются *гипотезами о коде*, подкреплёнными лишь тестами и код-ревью. С refinement они становятся следствием доказанной симуляции — ценой огромной инженерной работы: выбор представления состояния, доказательство шагов симуляции, учёт ошибочных путей, иногда — переписывание кода «под доказательство».

Есть и обратная сторона обещания. Refinement не делает модель богаче: он переносит *уже доказанное*. Если в модели нет гонок, уточнение не докажет их отсутствие в ядре — оно лишь потребует, чтобы конкурентность либо была добавлена в S, либо исключена допущениями («последовательное исполнение», «один поток владеет таблицей»). Честность refinement начинается с честности допущений.

10.3. Почему это трудно именно для микроядра

Микроядро Zircon — не идеальный объект для «полного seL4-стиля» в рамках статьи. Код написан с расчётом на производительность и совместимость: сложные структуры данных, ручное управление памятью, взаимодействие с драйверами и слоями вроде Starnix. Формальный refinement требует либо существенной *перестройки* кода под доказательство (как в seL4), либо инструментов, натягивающих логику на существующий C (AutoCorres [9], VST [10] и др. — инструменты, позволяющие доказывать свойства C-кода, не переписывая его целиком на функциональный язык), либо сужения цели до крошечного фрагмента.

Кроме того, сам объект уточнения нужно выбрать честно. Уточнять «всё ядро» к учебной модели бессмысленно: модель намеренно беднее (нет полного набора rights, нет конкуренции, MMIO введён как контрпример, а не как полная модель железа). Имеет смысл говорить о refinement *вырезки*: например, путь `channel_write/read + vmo_write` в изоляции, на последовательном исполнении, с явно оговорёнными допущениями о памяти.

Именно здесь MMIO-контрпример из теории `Zircon_MMIO` играет методическую роль: он напоминает, что даже идеальный IPC-путь не спасает, если рядом существует обход через привилегированную запись в ту же память. Refinement «только syscall» молчит о таких обходах, пока они не включены в отношение наблюдаемости. Поэтому горизонт уточнения — не лозунг «докажем всё ядро», а аккуратный выбор *какие* наблюдения мы считаем релевантными для переноса инварианта.

10.4. Место экстракции на пути к refinement

Экстракция Isabelle → OCaml не заменяет refinement к Zircon. Она решает другую, более скромную, но методически важную задачу: показывает, что определения модели *исполнимы* и что конкретные сценарии (*example_scenario*, пустая маска, алгебра *reduce_rights_wf*) дают ожидаемые значения не только в доказательстве `by simp`, но и при вычислении кодогенератором. Это — мост между этажами 1 и 2. Мост между этажами 2 и 3 остаётся отдельным исследовательским проектом; в обзорной работе достаточно ясно обозначить его цену и обещание, не имитируя завершённость.

Практический смысл такого моста для читателя-практика прост. Пока утверждение живёт только как лемма, легко заподозрить «магию тактик». Когда то же утверждение печатается как `True` командой `value [code]`, а затем подтверждается внешним OCaml, подозрение смещается: либо модель действительно считает так, либо ошибка общая для доказательства и генератора — и тогда её ловят расхождением с интуицией сценария. Это ещё не доказательство корректности ядра, но уже *воспроизводимый эксперимент* над спецификацией.

Иными словами: ниже мы спускаемся на этаж исполняемой спецификации и проверяем утверждения вычислением. Читатель при этом помнит, что до промышленного ядра ещё один — самый трудный — пролёт лестницы.

11. От доказательств к исполняемому коду

Формальные доказательства в Isabelle/HOL устанавливают свойства *модели*. Следующий шаг — связать их с исполняемой программой *той же* семантики (этаж 2 выше), не

претендуя ещё на бинарный Zircon. Isabelle предоставляет генератор кода, транслирующий подходящие спецификации в OCaml, Haskell, SML или Scala, а также команды `value` и `eval` для проверки конкретных выражений.

11.1. Принцип генерации

Используется мелкое встраивание (*shallow embedding*): логические константы и типы отображаются напрямую на сущности целевого языка (натуральные числа, множества, записи), без промежуточной модели синтаксиса. Сгенерированная программа реализует систему переписывания терминов; каждый шаг вычисления допускает *симуляцию* в логике (доказательство того, что конкретный результат кодогенератора соответствует формальному определению), что обеспечивает частичную корректность экстракции относительно выбранной спецификации кода.

В рассматриваемой модели автоматически транслируются, в частности, *create_vmo*, *zx_channel_write*, *zx_channel_read*, *transfer_via_channel*, *zx_handle_duplicate*, *zx_handle_close* и *vmo_write*.

11.2. Экспорт в OCaml

В сопровождающей теории достаточно указать экспорт:

```
export_code
  empty_state init_process create_channel_pair create_vmo
  zx_channel_write zx_channel_read transfer_via_channel
  vmo_write vmo_content lookup_handle
  zx_handle_duplicate zx_handle_close
  example_setup example_rights_a example_channel_rights
  example_exec_ok example_empty_exec_ok
  example_scenario example_empty_rights_ok
  in OCaml module_name Zircon file "generated/Zircon.ml"
```

Результат — модуль с сигнатурами вида

```
module Zircon : sig
  type 'a system_state_ext   (* в листингах ниже: system_state *)
  val create_channel_pair :
    nat -> nat -> right set -> right set ->
      unit system_state_ext ->
        (nat * (nat * (nat * unit system_state_ext))) option
  val zx_channel_write :
    nat -> nat -> nat -> right set ->
      unit system_state_ext -> unit system_state_ext option
  val zx_handle_duplicate :
    nat -> nat -> right set ->
      unit system_state_ext ->
        (nat * unit system_state_ext) option
  val vmo_write :
    nat -> nat -> nat ->
      unit system_state_ext -> unit system_state_ext option
  val example_exec_ok : bool
  val example_empty_exec_ok : bool
end
```

Идентификаторы — `nat` (Zarith), множества прав — конечные множества кодогенератора. Полный тип состояния — `unit system_state_ext` (расширяемый record Isabelle; внутри лежат и `channels`, и `channel_peer`). В сокращённых фрагментах ниже иногда пишем `system_state` для краткости.

11.3. Проверка утверждений: `value` и `eval`

Пока экстракт ещё не покинул Isabelle, те же константы можно *вычислить* прямо в среде разработки (PIDE — интегрированная среда Isabelle/jEdit). Команда `value [code]` запускает кодогенератор и печатает результат; *тактика* (встроенная процедура доказательства) `eval` использует тот же механизм, чтобы *доказать* равенство конкретному значению. Для учебных сценариев это даёт третий взгляд на одно и то же утверждение: лемма (`by simp`), вычисление (`value`), исполняемый OCaml (`test_code`).

В теории `Zircon_Code.thy`:

```
value [code] example_exec_ok
value [code] example_empty_exec_ok
value [code] example_scenario
value [code] example_empty_rights_ok
value [code]
  "reduce_rights_wf {ZX_RIGHT_WRITE}
     {ZX_RIGHT_READ, ZX_RIGHT_WRITE, ZX_RIGHT_TRANSFER}"

lemma example_exec_ok_by_eval: "example_exec_ok" by eval
lemma example_empty_exec_ok_by_eval: "example_empty_exec_ok" by eval
lemma example_scenario_by_eval: "example_scenario" by eval
lemma example_empty_rights_ok_by_eval: "example_empty_rights_ok" by eval
lemma example_wf_reduce_by_eval:
  "reduce_rights_wf {ZX_RIGHT_WRITE}
     {ZX_RIGHT_READ, ZX_RIGHT_WRITE, ZX_RIGHT_TRANSFER}
   = {ZX_RIGHT_READ, ZX_RIGHT_WRITE}"
  by eval
```

Ожидаемые результаты: оба сценария — `True`; замкнутое сокращение запроса {`WRITE`} против {`READ`,`WRITE`,`TRANSFER`} даёт {`READ`,`WRITE`} (зависимость WRITE⇒READ сработала). Расхождение `value`/`eval` с леммой `by simp` немедленно сигнализировало бы о проблеме в setup кодогенерации или в определении константы.

11.4. Проверка сгенерированного кода

Помимо внешних фреймворков модульного тестирования (OUnit, Alcotest) Isabelle предоставляет команду `test_code`, которая компилирует сгенерированный код *внешним* компилятором OCaml и вычисляет заданное выражение уже вне Isabelle:

```
test_code "example_exec_ok" in OCaml
test_code "example_empty_exec_ok" in OCaml
```

Константа *example_exec_ok* кодирует тот же сценарий, что и доказанная лемма *example_scenario_tru* пара каналов, VMO, передача только `READ`, move у отправителя, отказ `vmo_write` у получателя. В данной работе команды `value`/`eval` и `test_code` согласованы при сборке сессии; дополнительно автономный драйвер `generated/test_example.ml` подтверждает оба флага `= true` после компиляции `Zircon.ml`.

11.5. Инженерная врезка: что видно в OCaml

Инженерная врезка. Сторожа политики — запись в VMO, *req* ⊆ источника, замыкание прав, очередь пира и move — читаются в `Zircon.ml` как ветки `None`/`Some`. Ниже — что это даёт инженеру и что ломается, если ветку убрать.

Ниже приведены фрагменты сгенерированного модуля, где семантика capability видна глазами инженера. Это не полный дамп: кодогенератор добавляет прослойки для сравнения записей, обновления частичных функций и множеств; возможны штатные warning'и OCaml (неисчерпывающий match) — артефакт экстракции, не дефект модели. Фрагменты сокращены, но по смыслу совпадают с теорией; полный файл — `generated/Zircon.ml` после `isabelle build`.

Сторож записи: без `WRITE` пути нет.

```
(* Zircon.ml: vmo_write *)
let vmo_write pid hid content s =
  match lookup_handle pid hid s with
  | None -> None
  | Some h ->
      if obj_type (h_obj h) <> VMO then None
      else if not (mem ZX_RIGHT_WRITE (h_rights h))
           then None   (* <-- отказ: нет WRITE *)
      else Some (update_vmo_store ...)
```

Что это даёт: целостность содержимого VMO становится *свойством функции*, а не комментарием в документации syscall. В сценарии `example_exec_ok` получатель после передачи только `READ` вызывает `vmo_write` — и получает `None`; константа сценария требует именно этого отказа (`is_none (vmo_write …)`).

Что было бы без этого: можно «доказать» no-escalation на уровне масок handles и всё равно позволить запись в store любому, у кого есть дескриптор. Левая рука модели (права) разошлась бы с правой (мутация объекта) — классический разрыв спецификации и исполняемого поведения, который тесты ядра ловят поздно, а экстракт — сразу, если сценарий требует `None`.

Сторож делегирования: subset, замыкание, пир.

```
(* Zircon.ml: схема ветвлений zx_channel_write *)
(* ... WRITE на канал, TRANSFER у payload, payload ≠ сам канал ... *)
if not (less_eq_set req_rights (h_rights h_pay)) then None
else match channel_peer s (obj_id (h_obj h_ch)) with
  | None -> None
  | Some peer ->
      match channels s peer with
      | None -> None
      | Some q ->
          let actual = reduce_rights_wf req_rights (h_rights h_pay) in
          (* enqueue cap на peer; move pay_hid → None *)
```

Два независимых слоя. Сначала *req* $\subseteq h_rights$: запрос сверх исходных прав даёт `None` (аналог `ZX_ERR_INVALID_ARGS`) — «попросить {`WRITE`} при источнике без него» на этаже 2 уже невозможно. Затем `reduce_rights_wf`: при допустимом запросе замыкание может добавить `READ` к {`WRITE`}, но только если `READ` уже был у источника (*rights_wf* из `state_wf`; сам экстракт этого не проверяет). Сообщение уходит в очередь *пира*, не в свою.

Что было бы без subset: исполняемый эталон молча брал бы □ и «прощал» завышенный запрос. Без `close_rights` мог бы появиться {`WRITE`} без `READ`. Без `channel_peer` оба конца делили бы одну очередь — self-read.

Сторож move: дескриптор отправителя обнуляется.

```
(* внутри успешной ветки zx_channel_write *)
let actual = reduce_rights_wf req_rights (h_rights h_pay) in
let cap    = Handle_ext (h_obj h_pay, actual, ()) in
let ht'    = fun_upd ht pay_hid None in  (* move *)
```

```
let s1     = set_table pid ht' s in
(* channels_update: peer ↦ q @ [cap] *)
```

Handle_ext — конструктор экстракта для record handle (type_synonym capability = handle): третий unit-компонент — расширение record HOL, не тип Fuchsia. После успеха lookup_handle у отправителя по pay_hid даёт None (example_exec_ok).

Что было бы без move: два процесса держали бы capability на один объект после «передачи»; угроза утечки через забытый дескриптор осталась бы вне модели.

Сценарий как исполняемый контракт. Константа example_exec_ok — это не unit-тест, написанный «сбоку», а свёртка того же сюжета, что и лемма: канал, VMO со значением 42, передача {READ}, move, отказ записи 99, содержимое по-прежнему 42. Автономный драйвер сводится к двум строкам:

```
(* generated/test_example.ml *)
let ok_main = Zircon.Zircon.example_exec_ok in
let ok_empty = Zircon.Zircon.example_empty_exec_ok in
if ok_main && ok_empty then (
  print_endline "PASS: example_exec_ok = true";
  print_endline "PASS: example_empty_exec_ok = true";
  exit 0
) else (
  if not ok_main then print_endline "FAIL: example_exec_ok = false";
  if not ok_empty then print_endline "FAIL: example_empty_exec_ok = false";
  exit 1
)
```

Если завтра определение zx_channel_write «упростили» и забыли move или проверку WRITE, сборка сессии (by eval / test_code) и make test упадут *одновременно* с расхождением относительно прежней леммы — при условии, что лемму тоже придётся пересматривать. Именно эта связка «доказательство ↔ значение ↔ бинарник OCaml» и есть практический смысл этажа 2: не замена ядру, а невозможность тихо разъехаться модели и исполняемому эталону.

11.6. Сопоставление с традиционным тестированием

Таблица 4: Сравнение традиционных модульных тестов и проверки экстрагированного кода.

Аспект	Традиционные тесты	Тесты экстракта
Источник	ручная разработка	выражения из спецификации / лемм
Общность	отдельные примеры	случаи, покрываемые определением модели
Связь с верификацией	обычно отсутствует	тест — срез верифицированной модели
Ошибки генератора	не затрагиваются	могут быть обнаружены расхождением

Эмпирическая проверка экстракта не заменяет доказательств: она дополнительно связывает модель с исполняемым артефактом и повышает уверенность в настройке кодогенерации.

12. Дальнейшая работа

На уровне учебной модели закрыты онтология каналов с handles, move, пустые маски, решётка (зависимости в syscall-пути; join — иллюстрация), трассы (`state_wf`) и MMIO-контрпример. *Расширение полного syscall surface* промышленного Zircon намеренно оставляем за рамками обзорной статьи: это самостоятельный многолетний объём, сопоставимый с отдельным проектом ядра, а не с методическим текстом.

12.1. Уже выполненное в теориях

Ниже — карта артефактов сессии: какие именно утверждения доказаны, в какой теории живут и какую роль играют — ядро вклада или иллюстрацию. Простое перечисление имён файлов было бы бесполезно: читателю важно знать, что *именно* доказано.

Ядро вклада vs иллюстрации. *Ядро: channel_transfer_no_escalation*, *channel_write_requires_tran*
channel_write_moves_sender_handle, *vmo_write_denied_without_write_right*, *mmio_leak_witness_*
zx_handle_duplicate_no_escalation, *reduce_rights_wf_le_old*, *reachable_preserves_wf*,
reachable_transfer_no_escalation, *reachable_duplicate_no_escalation*. *Границы: transfer_without_w*
(без инварианта источника теорема ложна), *example_trace_reachable_wf* (учебный сценарий — достижимая трасса с корректным финалом). *Иллюстрации:* прочие решёточные следствия (`reduce_rights_*`), свидетели `example_*`, инженерная врезка OCaml, locale-интерпретация.

`Portable_Capability.thy`. Locale `portable_capability`: шесть аксиом attenuation, из которых выводится *reduce_wf_le_old*. Класс — монотонное сокращение масок с опциональным dependency closure; не универсальная модель прав.

`Zircon.thy`. Онтология (`handle` / `capability` = `handle`, каналы, VMO), move, duplicate/close, *reduce_rights/reduce_rights_wf*, основные syscall-определения; здесь же леммы ядра про передачу, duplicate и `vmo_write`, плюс свидетели *example_scenario_true*, *example_empty_rights_ok_t*

`Zircon_Lattice.thy`. Иллюстративный слой: отождествление сокращения со встречей (`inf`), следствия коммутативности/ассоциативности/монотонности, join как *combine_rights*, перенос no-escalation на *reduce_rights_wf*, мотивация *reduce_rights_may_break_wf*; плюс `interpretation` locale (*reduce_rights_wf_le_old_via_locale*).

`Zircon_Traces.thy`. Индуктивное *reachable* (метки включают `Duplicate` и `Close`), инвариант `state_wf` = `handles_wf` ∧ `channels_caps_wf` ∧ `peers_wf`, *reachable_preserves_wf* и security на трассах: *reachable_transfer_no_escalation* / *reachable_transfer_step_no_escalation* / *reachable_duplicate_no_escalation*; учебный сценарий как достижимая трасса (*example_trace_reach*

`Zircon_MMIO.thy`. Ядро вклада по обходам: *privileged_vmo_poke*, свидетель *mmio_leak_witness_tr* и разделение «шлюз `vmo_write` жив» / «системная изоляция store мертва» при наличии обхода (см. раздел о MMIO).

`Zircon_Code.thy`. Иллюстрация этажа 2: экстракция в OCaml, `value`/`eval`, `test_code`; исполняемые контракты *example_exec_ok*, *example_empty_exec_ok*.

12.2. Горизонт: refinement к вырезке ядра

Естественное продолжение — не расползание API, а *уточнение узкого пути* (см. раздел о refinement): взять опосредованный `channel_write/read + vmo_write`, зафиксировать допущения о последовательном исполнении и показать симуляцию между исполняемой спецификацией (OCaml-экстракт или идеализированный C) и фрагментом реализации. Полный Zircon при этом остаётся контекстом угроз и совместимости, а не объектом тотальной верификации. Цель такого уточнения — упрощённая версия той же задачи, которую seL4 решает целиком [4]: не покрыть всё ядро, а зафиксировать *вырезку* и показать симуляцию именно для неё. Разница с seL4 — не «мы так же строги, только короче», а масштаб спецификации и TCB: тот же *тип* свойства (no-escalation) на учебной вырезке, без refinement до бинарника. Границу мы обозначаем явно: то, что осталось за ней, осталось неверифицированным.

Порядок. Сначала укрепить этаж 2 (больше `value/eval`-свидетелей, свойства экстракта). Трассовая лемма `reachable_transfer_no_escalation` уже в сессии (`Zircon_Traces.thy`). Далее — минимальный C-фрагмент для частичного refinement без обязательства покрыть весь syscall surface.

13. Перенос методологии: от Zircon-фрагмента к родственным моделям

Настоящая работа — формализация *конкретного* фрагмента capability-делегирования в стиле Zircon. Помимо самой модели мы выделяем из неё переносимый интерфейс — монотонное ослабление прав с опциональным замыканием зависимостей. Инженер, разрабатывающий своё микроядро, не должен копировать *zx_channel_write* или *create_channel_pair*: имена, сигнатуры и онтология у каждого ядра свои. Но если механизм прав попадает в тот же *класс* — монотонное сокращение масок с опциональным замыканием зависимостей — структура доказательств из §3–§8 переносится почти без изменений. Задача раздела — зафиксировать этот класс, показать Isabelle-локаль `portable_capability` (теория `Portable_Capability.thy`, интерпретация в `Zircon_Lattice.thy`) и явно перечислить модели, для которых нужна *своя* формализация.

13.1. Три уровня, которые нельзя смешивать

Опыт формализации capability-механизма удобно раскладывается на три уровня, которые часто путают.

1. **Архитектура доказательства.** Общая схема: как права связаны с системными вызовами, как доказывается сохранение инварианта состояния на каждом шаге, как из пошаговых утверждений получается индукция по трассе, как предъявляется негативный свидетель (negative witness, §9) для обхода. Этот уровень *не зависит* от конкретных имён и сигнатур.

2. **Определения и инвариант.** Тип `rights`, отношение порядка, операции `reduce/close`, предикат `state_wf`, семантика конкретных переходов. Этот уровень *свой* у каждого ядра.

3. **Соглашения и нотация.** Имена лемм, порядок аргументов, формат syscall'ов, набор прав (`ZX_RIGHT_WRITE` и т. п.). Этот уровень *вообще не переносится* и не должен — иначе получается не модель нового ядра, а пародия на Zircon.

Практическое следствие: не начинать с имён лемм (уровень 3) — иначе доказательства не сходятся, потому что инвариант не продуман. Порядок уровней 1 и 2 зависит от того, о чём речь. *В голове* полезно сначала набросать архитектуру доказательства (какие леммы нужны, какой инвариант должен сохраняться) — это уровень 1 как каркас. *В коде* Isabelle сначала выписывают определения и `state_wf` (уровень 2), затем инстанцируют locale (уровень 1 как аксиомы) — иначе нечему удовлетворять аксиомам. Маршрут §13.7 и рис. 4 идут именно по коду: определения → locale → операция → трасса.

13.2. Портируемый скелет: интерфейсные леммы

Центральное наблюдение: доказательства no-escalation в §8.5 не используют ничего специфического про Zircon, кроме свойств пары (*reduce*, *close*) и предиката корректности масок. Если сформулировать эти свойства как аксиомы — получается переносимая рамка. В Isabelle это удобно оформить через `locale`.

Прежде чем читать код, поясним инструменты Isabelle, которые здесь используются, — читателю, знакомому с функциональным программированием, но не с Isabelle, они могут быть незнакомы.

`locale` (локаль) — именованный контекст с аксиомами. `locale` в Isabelle — это не «модуль» и не «класс» в смысле ООП, а скорее шаблон теории с открытыми параметрами и *аксиомами на них*. Внутри `locale` можно вводить определения и доказывать леммы; все они неявно зависят от аксиом и верны для *любого* набора параметров, удовлетворяющего этим аксиомам. Это позволяет сформулировать один раз «общий скелет» и потом многократно переиспользовать его для разных конкретных систем.

`interpretation` (интерпретация) — подстановка конкретных значений. `interpretation` — это команда, которая подставляет в `locale` конкретные значения параметров и требует доказать, что аксиомы `locale` для них выполнены. После успешной интерпретации все леммы, доказанные внутри `locale`, становятся доступны для подставленных значений *бесплатно*: их не нужно доказывать заново. Именно так ниже Zircon подставляется в `portable_capability`: *reduce* := `reduce_rights`, *close* := `close_rights`, *le* := (⊆), *wf* := `rights_wf`. После доказательства шести аксиом лемма locale *reduce_wf_le_old* становится доступна как `zircon_portable.reduce_wf_le_old`. Одна короткая выводная лемма — *reduce_rights_wf_le_old_via_locale* — связывает её с уже знакомым именем; отдельного содержательного доказательства для этого не требуется, только равенство `zircon_portable.reduce_wf = reduce_rights_wf`. (В `Zircon.thy` есть и прямое, не через locale, доказательство *reduce_rights_wf_le_old* — то же свойство, другой путь.)

```
locale portable_capability =
  fixes reduce :: "'r ⇒ 'r ⇒ 'r"       (* сокращение прав *)
  fixes close  :: "'r ⇒ 'r"            (* замыкание зависимостей *)
  fixes wf     :: "'r ⇒ bool"          (* корректность маски *)
  fixes le     :: "'r ⇒ 'r ⇒ bool"  (infix "⊑" 50)
  (* для Zircon: ⊑ есть ⊆ *)
  assumes reduce_le_old:
            "reduce r s ⊑ s"
  assumes reduce_greatest:
            "c ⊑ r ⟹ c ⊑ s ⟹ c ⊑ reduce r s"
  assumes close_idem:
            "close (close r) = close r"
  assumes close_infl:
            "r ⊑ close r"
  assumes close_mono:
            "r ⊑ s ⟹ close r ⊑ close s"
```

```
  assumes close_fix_wf:
            "wf r ⟷ close r = r"
begin

definition reduce_wf :: "'r ⇒ 'r ⇒ 'r" where
  "reduce_wf req src = close (reduce req src)"

lemma reduce_wf_le_old:
  assumes "wf src"
  shows "reduce_wf req src ⊑ src"
  (* close_mono + close_fix_wf + reduce_le_old *)
end
```

Обратите внимание на структуру: `reduce_le_old` — это «сокращение не превосходит источник»; `reduce_greatest` — характеризация как наибольшей нижней грани; три свойства `close` — идемпотентность, инфляция, монотонность; `close_fix_wf` — характеризация wf-масок как неподвижных точек. Всё вместе даёт *reduce_wf_le_old* в три строки. Locale живёт в теории `Portable_Capability.thy` (см. комментарий к `le` в листинге).

Именно эта лемма была «главной» в §8.5.1. Как только она сформулирована в locale, становится видно, что никакого парадокса не было: замыкание *безопасно*, если (а) оно монотонно, (б) оно расширяет маску, (в) корректная маска — его неподвижная точка. Парадокс возникал лишь потому, что мы работали в конкретной реализации и смотрели на отдельные леммы, а не на аксиомы.

Инженеру достаточно доказать эти шесть свойств для *своих reduce* и *close* — и всё, что следует из `reduce_wf_le_old`, получается бесплатно. В частности, для любой transferring-операции, использующей `reduce_wf`, no-escalation доказывается одинаково независимо от того, как устроены права — *внутри этого класса*.

Zircon как интерпретация locale. Модель статьи — `interpretation` в `Zircon_Lattice.thy`: `'r = right set`, порядок $\sqsubseteq$ — это $\subseteq$, *reduce* — пересечение $\cap$, *close* — замыкание по зависимостям из §8.5, *wf* — `rights_wf`. Все шесть аксиом доказаны; `Zircon_Lattice.thy` *интерпретирует* `portable_capability` на определениях из `Zircon.thy`, а не задаёт locale заново. Это не refinement из глоссария и §10 (симуляция реализации спецификацией), а подстановка параметров locale. Лемма *reduce_rights_wf_le_old_via_locale* — прямой след этой интерпретации.

13.3. Что должен определить инженер

Ниже — минимальный список того, что нельзя унаследовать и что придётся сформулировать заново. Порядок важен: от абстрактного к конкретному.

1. **Тип прав и порядок.** В Zircon это `set right` с $\subseteq$. У вас может быть решётка с $\sqsubseteq$ другого вида, декартово произведение, `map tag atom`, частично упорядоченная структура. Единственное требование — чтобы порядок был частичным и в нём существовала нижняя грань для пар, участвующих в *reduce*.

2. **Операция сокращения.** В Zircon это $\cap$. У вас может быть «пересечение с учётом приоритетов», «сужение по маске», «комбинация тегов». Нужно доказать `reduce_le_old` и `reduce_greatest`.

3. **Замыкание зависимостей (если есть).** В Zircon: `WRITE` $\to$ `READ`, `EXECUTE` $\to$ `READ`. У вас могут быть другие зависимости или их может не быть вовсе (тогда *close* $=$ *id* и три аксиомы тривиальны).

4. **Предикат `wf`.** В Zircon: «маска замкнута по зависимостям». У вас может быть дополнительное условие: «не содержит mutual exclusive флагов», «согласована с уровнем иерархии», «не превышает квоту». Ключевое — `close_fix_wf` должен быть *iff*: одной импликации *wf* $r \Longrightarrow$ *close* $r = r$ достаточно для *reduce_wf_le_old*, но обратная (*close* $r = r \Longrightarrow$ *wf* r) нужна, чтобы класс wf-масок в точности совпадал с множеством неподвижных точек замыкания. Без неё «замкнутость по зависимостям» перестаёт быть полной характеризацией `rights_wf`, и часть алгебраических следствий в §8 теряет опору. Содержательно *первая* импликация (*wf* $r \Longrightarrow$ *close* $r = r$) говорит, что замыкание не изобретает прав «из воздуха»: у корректной маски оно возвращает её саму. Обратная нужна для полноты: всякая неподвижная точка замыкания — уже wf-маска.

5. **Инвариант состояния `state_wf`.** В Zircon: `handles_wf` $\wedge$ `channels_caps_wf`. У вас — свой предикат на состоянии. Требование: он должен быть *предикатом на состоянии*, а не проверкой «на входе syscall», и каждая операция должна его сохранять.

6. **Передающие операции (attenuation path).** В Zircon — *zx_channel_write* и *zx_handle_duplicate*; у родственного ядра типичны `send`, `copy`, `derive`. Для каждой такой операции:

   - `op_no_escalation : h_rights dst` $\leq$ `h_rights src`;
   - `op_respects_request : h_rights dst` $\leq$ `close req`;
   - (желательно) `op_rights_eq : h_rights dst = reduce_wf req (h_rights src)`.

7. **Чеканка прав (minting) и отзыв (revoke) — отдельный слой.** Создание объектов с начальной маской и отзыв прав *не* покрываются `portable_capability`: нужны собственные аксиомы/инварианты (кто вправе mint'ить; что остаётся после revoke). Поясним термины. *Mint* — создание новой capability с заданной (урезанной) маской из произвольного источника, а не «наследованием» от уже существующей. *Revoke* — отзыв ранее выданной capability, то есть удаление её у того, кому она была передана. `Mint` принципиально *не* покрывается `portable_capability`: он может создать маску, которой у источника не было. `Revoke` тоже вне класса, но по другой причине — см. ниже. В нашей модели minting за рамками рассмотрения (см. §3.1); revoke отсутствует.

   Подчеркнём: это не «недосмотр», а структурный выбор класса. Класс `portable_capability` описывает операции *ослабления* маски при передаче или копировании: новое право должно быть выводимо из уже имеющегося у источника *в той же транзакции*. *Revoke* меняет права у процесса, который в текущей attenuation-транзакции не участвует, и потому не укладывается в аксиомы `reduce_le_old` и `close_infl` — не потому, что «даёт эскалацию» (отзыв прав никому не добавляет), а потому, что это другой жизненный цикл capability. В seL4 динамический рекурсивный `revoke` потребовал отдельной инфраструктуры доказательств именно по этой причине [4]. Расширение модели до поддержки `revoke` — это не «дописать ещё одну лемму», а перейти в другой математический класс. Без этого слоя картина «жизненного цикла права» неполна — но неполна она честно, а не по упущению.

8. **Обязательные проверки прав (authorization gates).** В Zircon их две на передачу: `ZX_RIGHT_WRITE` на дескриптор канала и `ZX_RIGHT_TRANSFER` на передаваемый дескриптор. У вас свои обязательные права вида `op_requires_right_X`. Эта сторона не менее важна, чем отсутствие эскалации: она гарантирует, что операция *не выполняется без нужного права*, а не только «не выдаёт лишнего».

9. **Поверхности обхода (bypass surfaces).** MMIO — лишь одна из возможных. Типичный список: DMA (в Zircon — через BTI/IOMMU, не «голый» доступ устройств к памяти), отладочные регистры, внутренние сокращения ядра (например, `sysctl` с записью в память пользовательского процесса), «привилегированный флаг» в системном вызове,

разделяемая память без capability. Для каждой такой поверхности — свой переход обхода и свой негативный свидетель по образцу §9.

Если пункты 1–4 заполнены, интерфейсные леммы доказываются автоматически внутри `portable_capability`. Пункты 5–6 и 8 — основная содержательная работа по attenuation-пути. Пункт 7 — отдельная архитектура. Пункт 9 уникален для каждого ядра и описывает то, что система *не* обещает.

13.4. Трассовый уровень: обязанность, а не опция

Пошаговые леммы — необходимое, но не достаточное условие безопасности. Свойство «ни при каком состоянии система не эскалирует права» относится к *трассам*, а не к отдельным шагам. Формально:

```
inductive reachable :: "system_state ⇒ system_state ⇒ bool" where
  refl: "reachable s s"
| step: "reachable s s1 ⟹ exec_step st s1 = Some s2
         ⟹ reachable s s2"

theorem reachable_preserves_wf:
  assumes "state_wf s" "reachable s s'"
  shows "state_wf s'"
```

Это доказывается индукцией по `reachable`, с одним подслучаем на каждую метку `step`. В модели статьи эта техника показана в §3.1, и там же построена трассовая теорема —

```
theorem reachable_transfer_no_escalation:
  assumes "state_wf s0" "reachable s0 s"
      and "transfer_via_channel src dst sch dch pay req s =
             Some (hid', s')"
      and "lookup_handle src pay s = Some h_src"
      and "lookup_handle dst hid' s' = Some h_dst"
  shows "h_rights h_dst ⊆ h_rights h_src"
```

Здесь трассовый уровень делает реальную работу, а не только упаковывает пошаговую лемму: *reachable_preserves_wf* даёт `state_wf`, из него — *rights_wf* источника, и пошаговая лемма применяется (зачем посылка и почему syscall её не проверяет — §3.1). Без трасс пришлось бы либо вернуть проверку в syscall (отход от Zircon), либо оставить предпосылку висеть. То же для метки `Transfer` и для duplicate.

Практическое замечание: индукция по трассам обычно даётся тяжелее, чем пошаговые леммы, потому что требует инварианта не только на конечном состоянии, но и на *всех* переходах вдоль трассы. Рекомендация — вводить вспомогательный предикат (у нас `state_wf` из трёх компонент) и доказывать его сохранение на каждом шаге и для каждой компоненты отдельно; тогда лемма о сохранении на трассе сводится к одной индукции по `reachable`. Побочный урок из этой работы: инвариант с симметричным условием (`peers_wf`: `peer a = b → peer b = a`) нельзя отдавать `simp`/`auto` в раскрытом виде вместе с посылкой — симплификатор зацикливается на условном правиле. Факты извлекаются деструктором (`peers_wfD`, через `blast`), а сохранение доказывается одной конгруэнцией (`peers_wf_cong`: связь концов не изменилась, очереди не исчезли, счётчик объектов не убыл).

13.5. Степени сходства и границы класса

Locale `portable_capability` покрывает *монотонную attenuation capability-масок* с опциональным dependency closure — не «любой механизм прав». Ниже — когда Zircon-текст

полезен инкрементально, а когда нужна отдельная модель.

Случай A: очень похоже. Ядро с таблицами дескрипторов, правами-подмножествами битов, каналами (или эквивалентным механизмом межпроцессного взаимодействия) и хранилищем памяти VMO-подобного типа. Тогда можно переиспользовать большую часть статьи: заменить имена (`zx_channel_write` → `ipc_send`) и сигнатуры, а решёточная часть, локаль и экстракция переносятся как есть.

Случай B: другие права, та же attenuation. Права — не множество битов, а, например, `map tag atom` с покоординатным порядком. Тогда `portable_capability` инстанцируется с этим порядком, `reduce` — покоординатный минимум, `close` — dependency closure по координатам (или `id`, если зависимостей нет; см. §13.3, п. 3). Меняется в основном §8; syscall-леммы §5–6 сохраняют форму.

Случай C: другая модель объектов. Вместо таблиц дескрипторов — capability space (пространство полномочий, seL4-стиль: CSpace, слоты, CNode), вместо каналов — IPC endpoints (концы каналов межпроцессного взаимодействия) со слотами. Тогда `state_wf` пишется заново (`cnode_wf`, `slot_wf`, …). Архитектура «интерфейс + инвариант + шаг + трасса + witness» остаётся; содержание §2–4 — своё. Полный seL4 — с типизированной памятью (*untyped*), чеканкой новых прав (*mint*), метками для различения копий (*badge*) и деревом рекурсивного отзыва (*revoke tree*), — сюда *не* входит целиком: это уже соседний класс с отдельной онтологией.

Случай D: параллелизм. Параллельные системные вызовы требуют уточнения под блокировку или семантику чередования — это исследовательская, а не инкрементальная работа. Zircon-фрагмент даёт семантику отдельного шага, но не рамку для параллелизма.

Прочие различия внутри класса. Отсутствие move (только copy) — no-escalation сохраняется, но ломается свойство «число держателей не растёт при передаче» (см. §5.3). Refcounting — расширение `state_wf`. Иерархия job → process — `hier_wf` и no-escalation на каждом уровне.

Вне класса: нужны свои модели. Следующие семейства *существуют* и важны, но `portable_capabil` на них не претендует — для каждого придётся строить отдельную формализацию (возможно, с другим набором аксиом и другим главным инвариантом):

- **ACL / DAC** — списки доступа и владельцы объектов, а не передаваемые маски-capabilities;
- **RBAC** — роли и назначения, а не attenuation по meet/closure;
- **Information-flow / noninterference** — свойства потоков данных, а не локальный no-escalation маски;
- **Полный seL4-стек** — untyped memory, mint/badge, recursive revoke, CSpace; пересечение с нашей темой есть, но каркас другой;
- **Spatial / temporal capabilities** — диапазоны адресов, сроки жизни, эфемерные права;
- **Concurrent capability-ядра** без сведения к атомарному шагу — см. случай D.

Итог: статья даёт *образец жанра* для attenuation-фрагмента в духе Zircon и переносимый locale для того же класса. Остальное — соседние задачи со своими моделями.

13.6. Минимальный жизнеспособный набор

Если у инженера мало времени — есть ровно четыре леммы, которые нужно доказать, чтобы получить содержательный результат.

1. `state_wf : system_state ⇒ bool` — предикат корректности.
2. `send_preserves_state_wf : send ... s = Some s' ⟹ state_wf s ⟹ state_wf s'` — сохранение инварианта на самой важной transferring-операции.
3. `send_no_escalation : send ... s = Some s' ⟹ h_rights dst ≤ h_rights src` — ключевая security-лемма на одном шаге.
4. `reachable_preserves_wf` — индукция по трассе.

Если эти четыре собираются в Isabelle, то: онтология продумана, инвариант состояния не пуст, главная security-лемма доказуема, трассовый уровень работает. Всё остальное (дубликаты, close, MMIO-контрпример, экстракция) — это обогащение и защита от конкретных архитектурных соблазнов, не обязательно на первом шаге.

13.7. Практический порядок работы

Напоследок — конкретный маршрут для инженера, начиная с нуля.

1. **Не писать утверждения и леммы Isabelle до того, как выписаны три вещи**: порядок на правах, операции `reduce`/`close`, определение `state_wf`. Это уровень 2 из §13.1; без него любая формализация рассыпается.
2. **Инстанцировать `portable_capability`**: доказать шесть аксиом для своих `reduce`, `close`, `wf`. Это уровень 1; после этого `reduce_wf_le_old` получается бесплатно.
3. **Взять одну transferring-операцию — самую главную.** Доказать сохранение `state_wf` и no-escalation. Одна операция, два доказательства, одно понимание.
4. **Ввести `reachable`**. Доказать `reachable_preserves_wf` индукцией по трассе.
5. **Итерировать по остальным операциям.** Каждая — это ещё одна пара лемм, по образцу шага 3.
6. **Трассовая часть до `reachable_transfer_no_escalation`**: разбор метки `Transfer` на reachable-состоянии. В нашем репозитории шаг выполнен (`Zircon_Traces.thy`); при переносе — воспроизвести по тому же образцу.
7. **Добавить negative witnesses** для каждого bypass'а. Без них карта безопасности неполна: вы не знаете, что ваша система *не* обещает.
8. **(Опционально) экстракция в OCaml и `test_code`** по образцу §11: превратить ключевые леммы в исполняемые контракты.
9. **(Опционально) refinement к вырезке C-кода**: уточнить узкий путь, не пытаясь покрыть всё ядро.

Порядок неслучаен. По нашей оценке, первые четыре шага дают основную долю практической ценности (эвристически ∼70%) и обычно требуют нескольких дней на свою реализацию; это не измерение, а ориентир. Шестой шаг — основнаясодержательная работа по трассам, тоже измеряется днями. Negative witnesses (шаг 7) — методический вклад, который часто пропускают, но именно он превращает формализацию в честный документ. Шаги 8–9 — следующий уровень сложности, сопоставимый с §10–12 статьи.

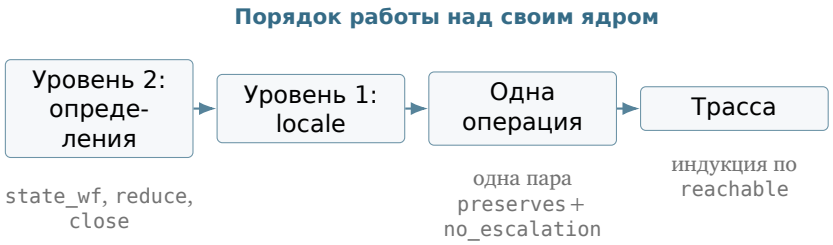


Рис. 4: Маршрут переноса. Слева — то, что инженер определяет сам. Справа — то, что доказывается по образцу статьи.

Что статья даёт, а что — нет. Наш фрагмент модели — это *пример жанра* ослабления прав (*attenuation capabilities*), а не шаблон «для любого ядра». Он показывает три вещи: компактную формализацию отсутствия эскалации, исполняемый мост к OCaml и негативный свидетель для обхода через MMIO. Что переносится *внутри этого класса* — так это локаль `portable_capability` и архитектура доказательства: их можно переиспользовать для других моделей того же класса, заменив лишь конкретные определения. Остальные модели прав существуют и требуют своих теорий.

Именно поэтому вклад статьи мы формулируем скромно: не как доказательство безопасности промышленного Zircon и не как совпадение имён с ним, а как *работающий Isabelle-интерфейс* для класса монотонного ослабления прав вместе с честно очерченными границами применимости.

14. Заключение

Построена учебная формализация фрагмента capability-модели в стиле Zircon и установлены машинно проверяемые свойства: отсутствие эскалации прав при передаче, сохранение ссылки на объект, необходимость `TRANSFER` при отправке и запрет записи в VMO без `WRITE`. Сокращение прав совпадает со встречей дистрибутивной решётки подмножеств; алгебраически no-escalation — следствие законов этой структуры, но *доказательный путь* главных security-лемм идёт напрямую в `Zircon.thy`, а решёточные варианты в `Zircon_Lattice.thy` остаются параллельной иллюстрацией (см. §5 и §8). Определения экстрагированы в OCaml; ключевые сценарии подтверждены тройным взглядом — леммой, вычислением `value/eval` и внешним `test_code`.

Отдельно обозначена проблематика refinement: доказанное на этаже модели не переносится автоматически на C++-ядро, пока не построена симуляция к вырезке реализации. Для обзорной статьи достаточно честно провести эту границу и укрепить мост к исполняемой спецификации, не имитируя полный syscall surface промышленного Zircon. Раздел 13 показывает, как ту же архитектуру доказательства перенести на *родственный* механизм attenuation-масок: через locale `portable_capability` (и явную границу класса — ACL, RBAC, IFC, полный seL4 и т. д. требуют своих моделей).

Полученный результат не следует интерпретировать как верификацию промышленного ядра Zircon: для задач такого масштаба необходимы усилия, сопоставимые с проектами класса seL4. Вместе с тем даже упрощённая модель не является «игрушкой»: она показывает, что инвариант отсутствия эскалации доказуем локально, без моделирования драйверов и полного syscall surface. В связке с документацией Fuchsia/Zircon [6, 5] такая учебная формализация даёт security-специалисту воспроизводимый язык спецификации и ранней проверки инвариантов делегирования прав — до того как компромиссы совместимости (драйверы, MMIO, слои вроде Starnix) начнут размывать архитектурную дисциплину.

Исходные материалы. Постоянная ссылка на репозиторий с теориями и экстрактом: https://github.com/redbeardster/zircon-article. Проверено на **Isabelle2025** и OCaml **4.14.1** с `zarith` **1.14** (`ocamlfind + zarith`). Сборка: `isabelle build -d . Zircon`. Сгенерированный модуль `generated/Zircon.ml`; проверка: `cd generated && make test`.

Автор: redbeardster, 2026.

Список литературы

[1] Tobias Nipkow, Gerwin Klein. *Concrete Semantics with Isabelle/HOL*. Springer, 2014. http://concrete-semantics.org/.

[2] Tobias Nipkow, Lawrence C. Paulson, Markus Wenzel. *Isabelle/HOL — A Proof Assistant for Higher-Order Logic*. Lecture Notes in Computer Science, vol. 2283. Springer, 2002. https://doi.org/10.1007/3-540-45949-9. Актуальная документация системы: https://isabelle.in.tum.de/documentation.html.

[3] Gerwin Klein et al. *seL4: Formal Verification of an OS Kernel*. In: SOSP '09. ACM, 2009. https://dl.acm.org/doi/10.1145/1629575.1629596.

[4] Gerwin Klein, June Andronick, Kevin Elphinstone, Toby Murray, Thomas Sewell, Rafal Kolanski, Gernot Heiser. *Comprehensive Formal Verification of an OS Microkernel*. ACM Trans. Comput. Syst., 32(1), 2014. https://doi.org/10.1145/2560537.

[5] Fuchsia documentation. *Rights*, *zx_channel_write* and *zx_channel_write_etc*. https://fuchsia.dev/fuchsia-src/reference/kernel_objects/rights, https://fuchsia.dev/fuchsia-src/reference/syscalls/channel_write, https://fuchsia.dev/reference/syscalls/channel_write_etc.

[6] Fuchsia documentation. *Handles* and Zircon kernel concepts. https://fuchsia.dev/fuchsia-src/concepts/kernel/handles.

[7] Fuchsia documentation. *Resource* (`ZX_RSRC_KIND_MMIO`) and *zx_resource_create*. https://fuchsia.dev/fuchsia-src/reference/kernel_objects/resource, https://fuchsia.dev/reference/syscalls/resource_create.

[8] Fuchsia documentation. *Routing Capabilities* (Starnix). https://fuchsia.dev/fuchsia-src/development/starnix/routing_capabilities.

[9] David Greenaway, Japheth Lim, June Andronick, Gerwin Klein. *Don't sweat the small stuff: formal verification of C code without the pain*. In: PLDI '14 / companion technical report on AutoCorres. https://trustworthy.systems/projects/TS/autocorres/.

[10] Andrew W. Appel et al. *Verified Software Toolchain (VST)*. https://vst.cs.princeton.edu/.